

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

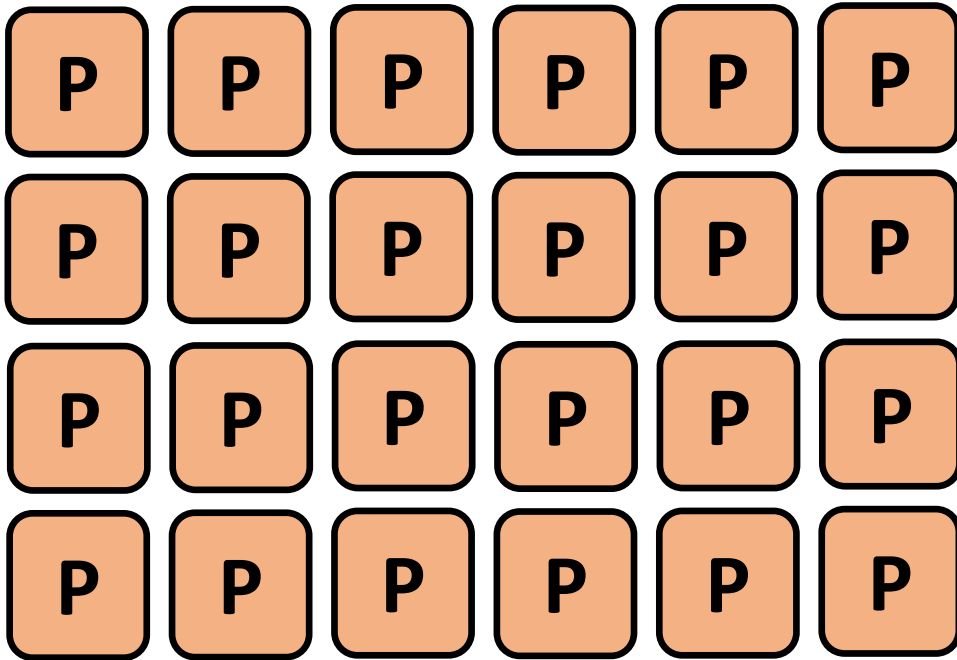
Heterogeneous Architectures and Accelerators

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan

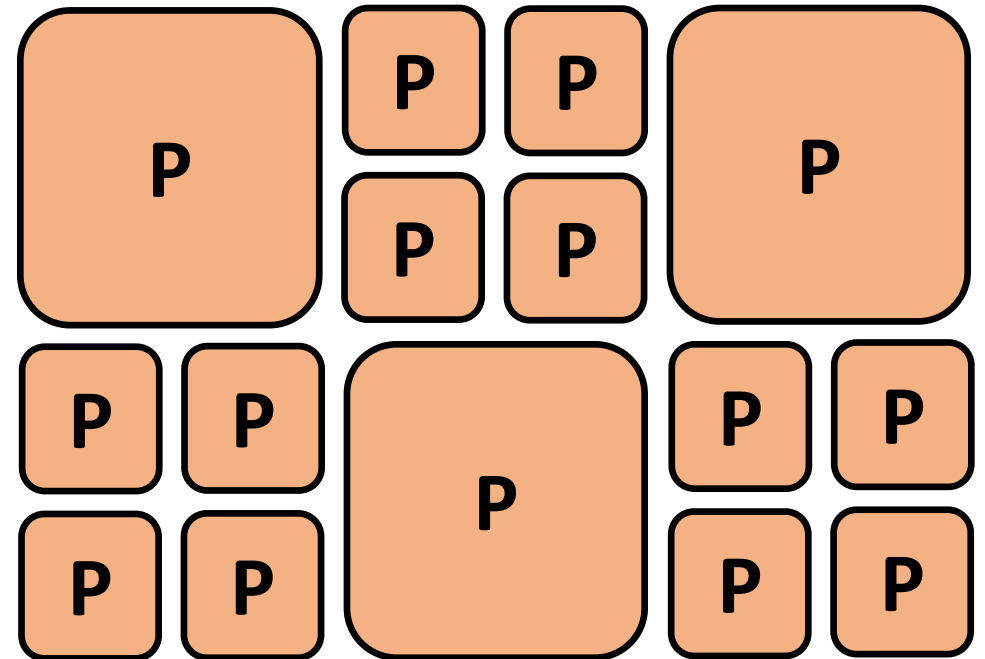
Which is Better?

homogeneous processor cores



VS

heterogeneous processor cores



Heterogeneous Amdahl's Law

M. D. Hill and M. R. Marty, *Amdahl's Law in the Multicore Era*, IEEE Computer, 2008

How do we design heterogeneous multicores for maximum speedup?

Heterogeneous Amdahl's Law

BCE = base-core equivalent, with performance of 1

Assume a given chip generation can contain n BCEs

- Includes L1 caches (but not L2 caches, interconnect, etc.)

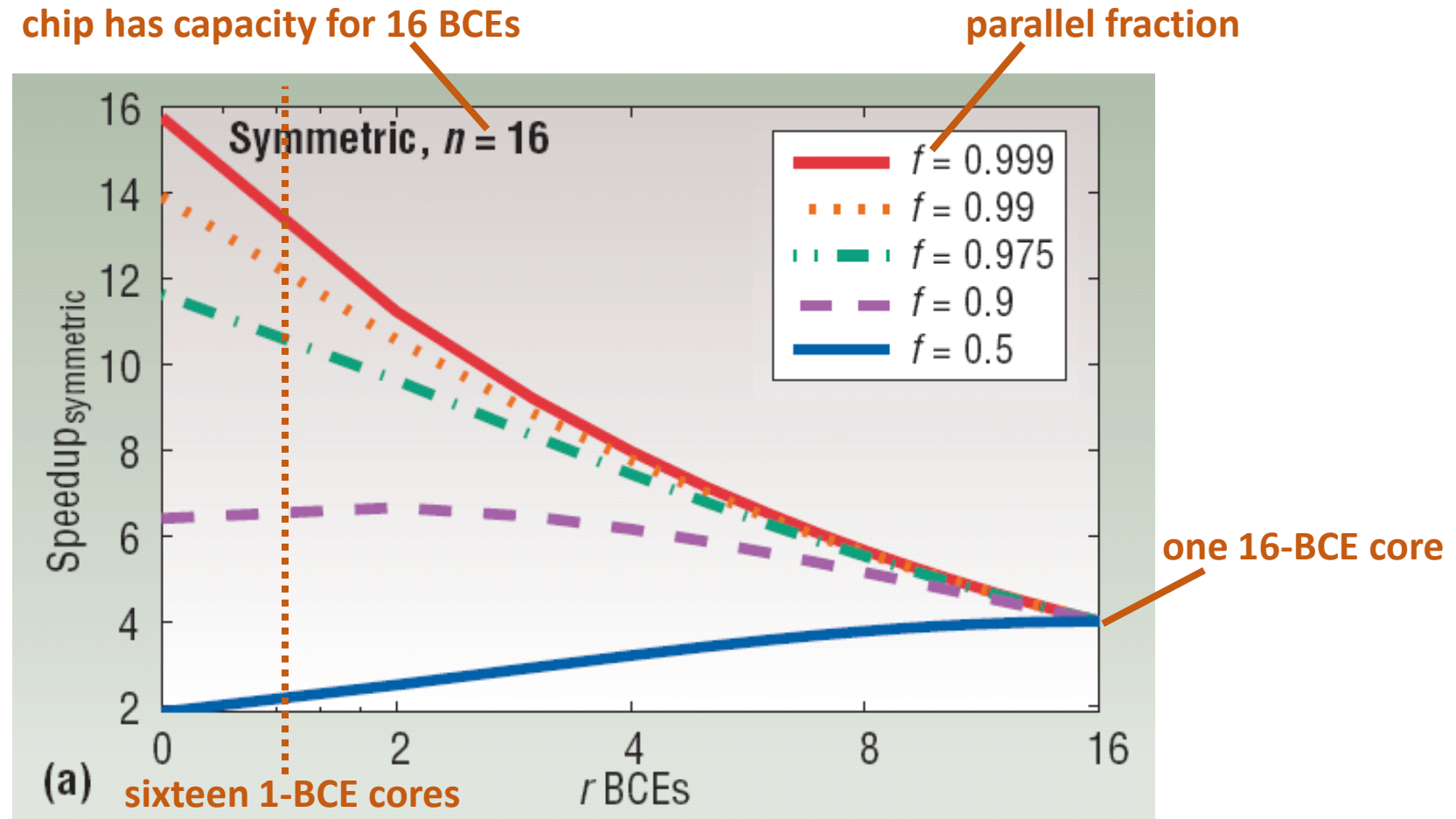
Assume that r BCEs can be replaced with a bigger r -BCE core

- e.g., Instead of two 1-BCE cores, could have one 2-BCE core

Assume that performance of r -BCE core is $\text{perf}(r) = \sqrt{r}$ (Pollack's Law)

- Performance improves slowly as you increase size of core

Heterogeneous Amdahl's Law – Symmetric

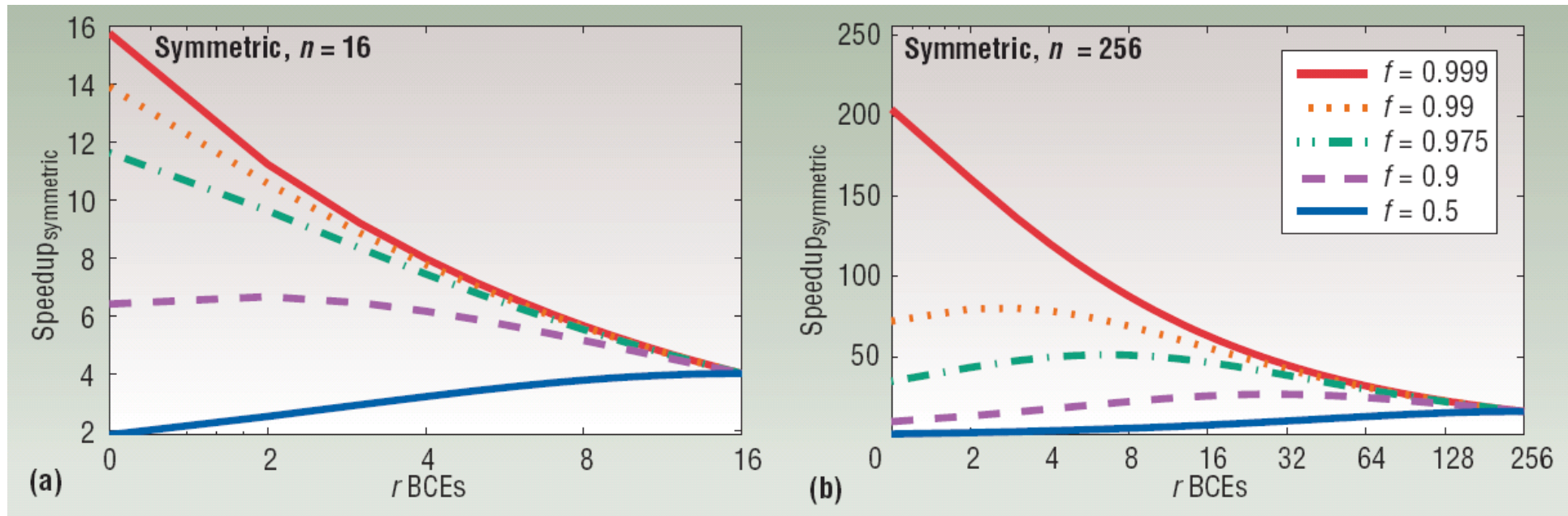


$$\text{Speedup}_{\text{symmetric}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f \cdot r}{\text{perf}(r) \cdot n}}$$

Heterogeneous Amdahl's Law – Symmetric

Takeaways:

- Maximizing parallel fraction is always critical
- Trade off improvements to parallel fraction vs. sequential fraction
- Optimal case is often when $r > 1$; we still need big cores

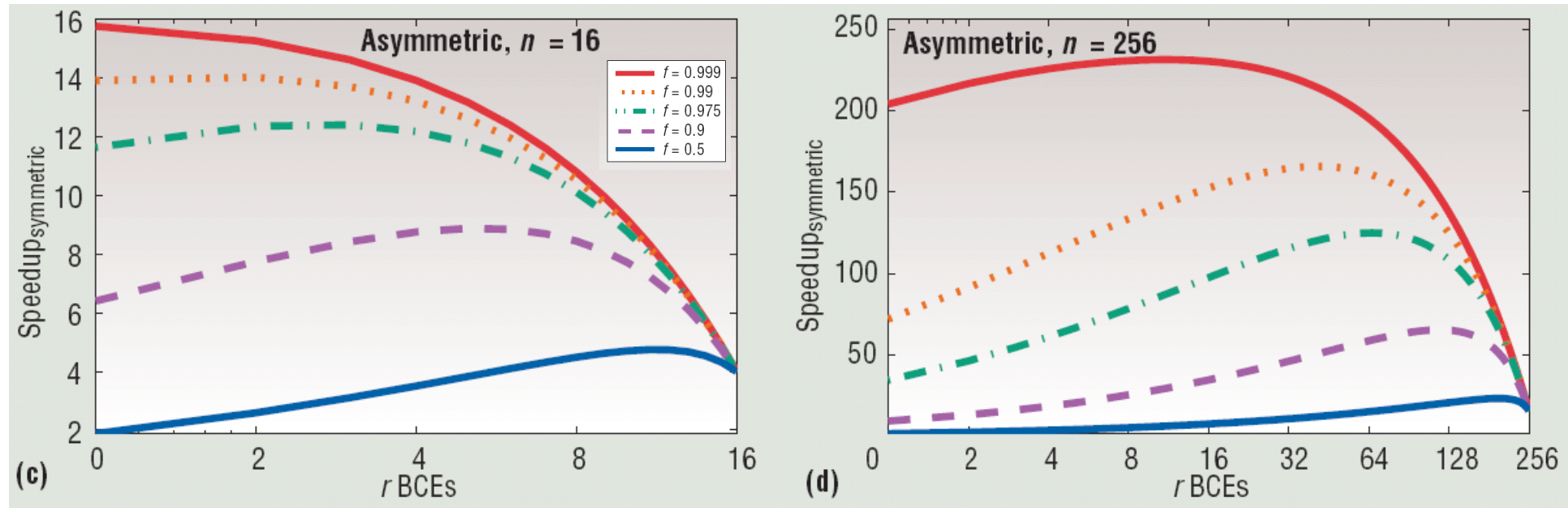


$$\text{Speedup}_{\text{symmetric}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f \cdot r}{\text{perf}(r) \cdot n}}$$

Heterogeneous Amdahl's Law – Asymmetric

Assumes one r -BCE core and the remaining $n-r$ cores are 1-BCE

- Equal or better performance than symmetric, since balances sequential and parallel
- Though must deal with scheduling challenges

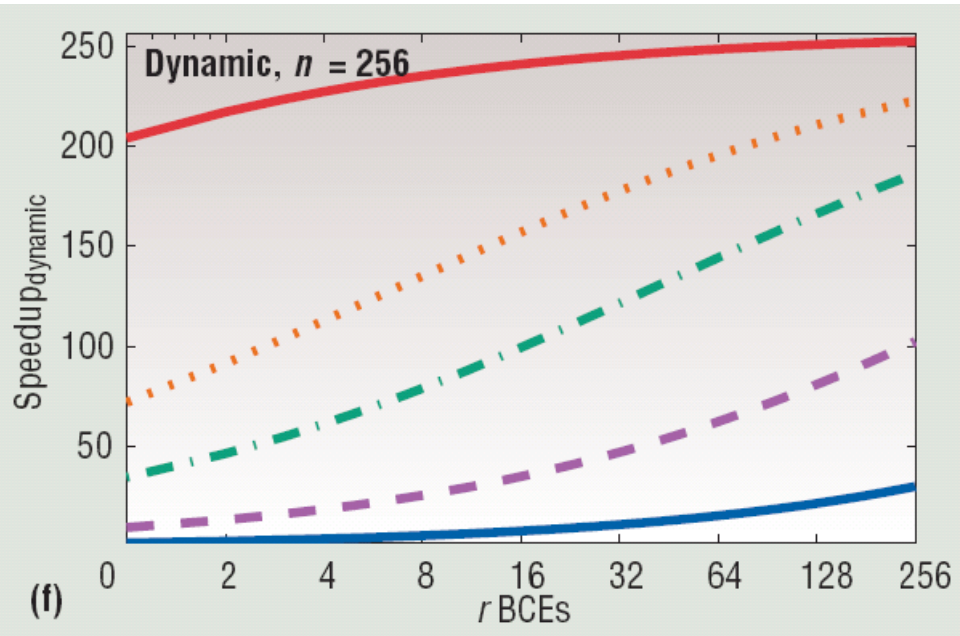
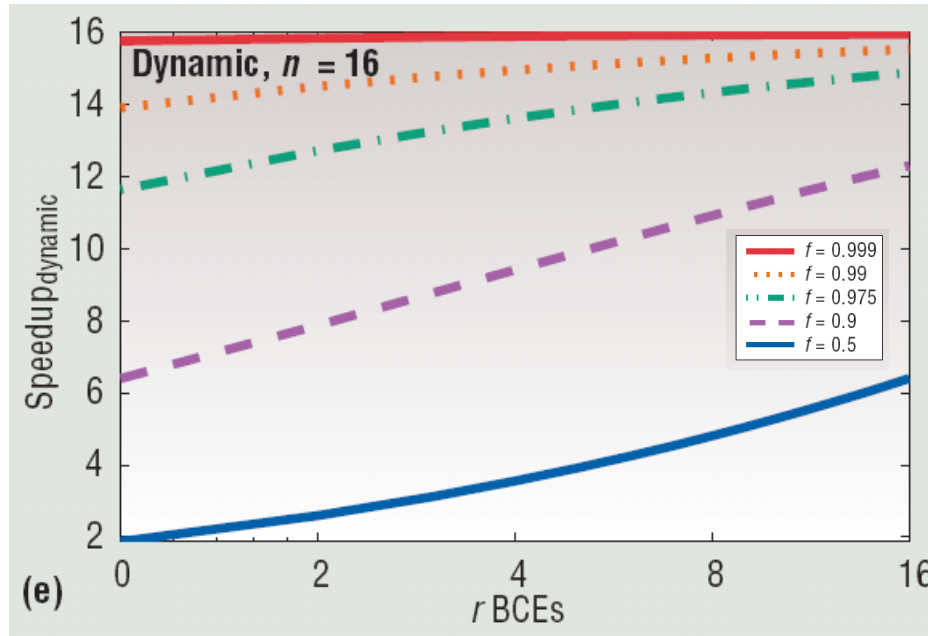


$$\text{Speedup}_{\text{asymmetric}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f}{\text{perf}(r)+n-r}}$$

Heterogeneous Amdahl's Law – Asymmetric

Assumes one dynamic r-BCE core and the remaining n-r cores are 1-BCE

- Dynamic core can serve as r-BCE for sequential as well as r 1-BCEs for parallel
- Equal or better performance than asymmetric, since best of both worlds
- Always better to have larger dynamic core



$$\text{Speedup}_{\text{dynamic}}(f, n, r) = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f}{n}}$$

Heterogeneous Amdahl's Law – Accelerators

E. S. Chung, P. A. Milder, J. C. Hoe and K. Mai, *Single-Chip Heterogeneous Computing: Does the Future Include Custom Logic, FPGAs, and GPGPUs?*, MICRO, 2010

What if we take accelerators into account?

Heterogeneous Amdahl's Law – Accelerators

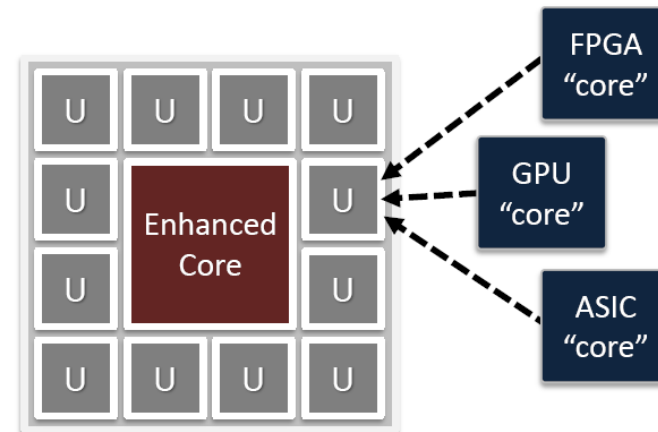
BCE = base-core equivalent, with performance of 1 and power of 1

Define U-core as BCE-equivalent area of FPGA/GPU/ASIC

- U-cores can only execute parallel kernels (e.g., matrix-matrix multiplication)
- U-core has performance μ and power ϕ relative to 1-BCE

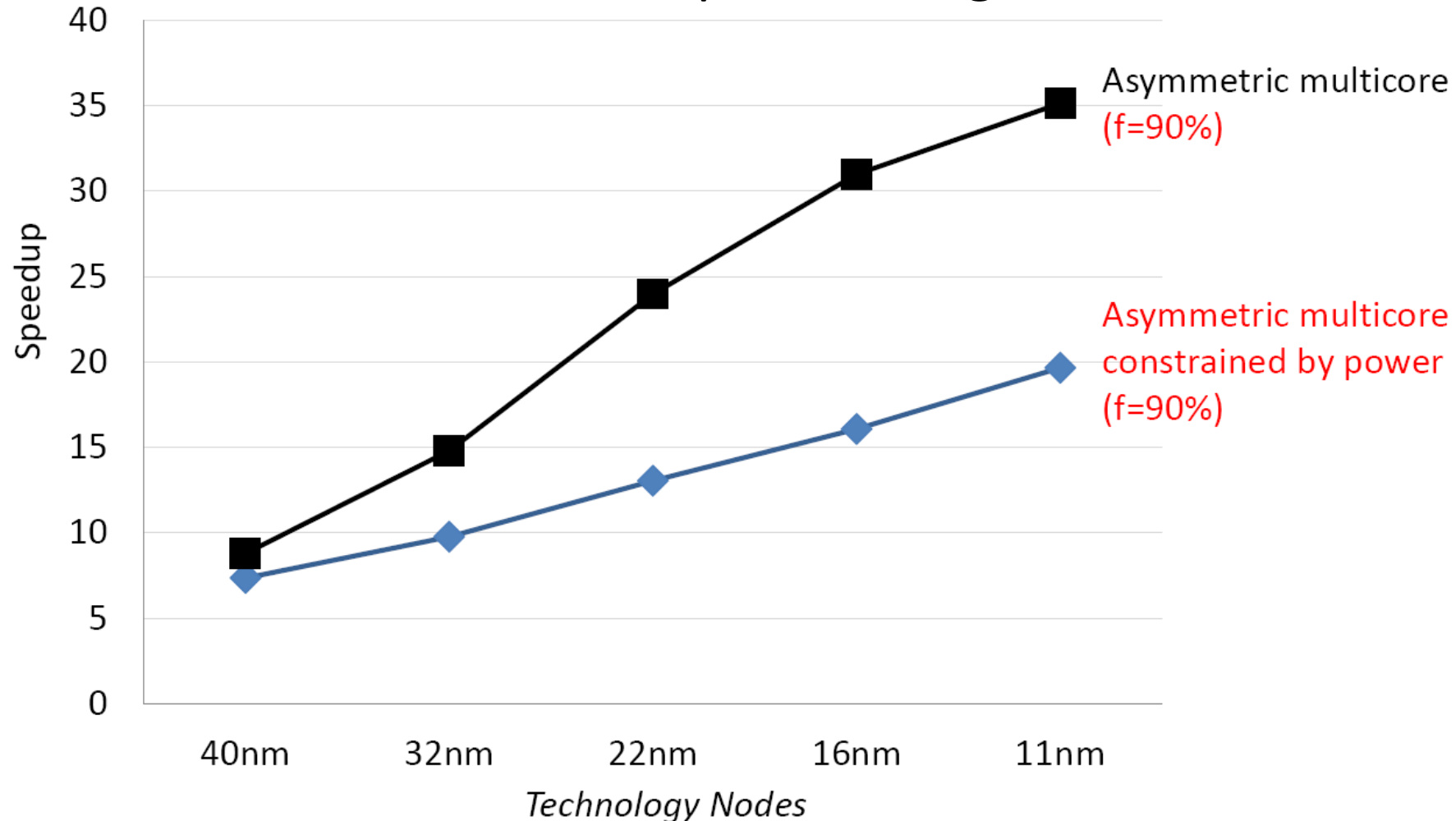
Assume one r-BCE core and the remaining n-r cores are U-cores

$$Speedup = \frac{1}{\frac{1-f}{perf_{enh}(r)} + \frac{f}{\mu \times (n-r)}}$$



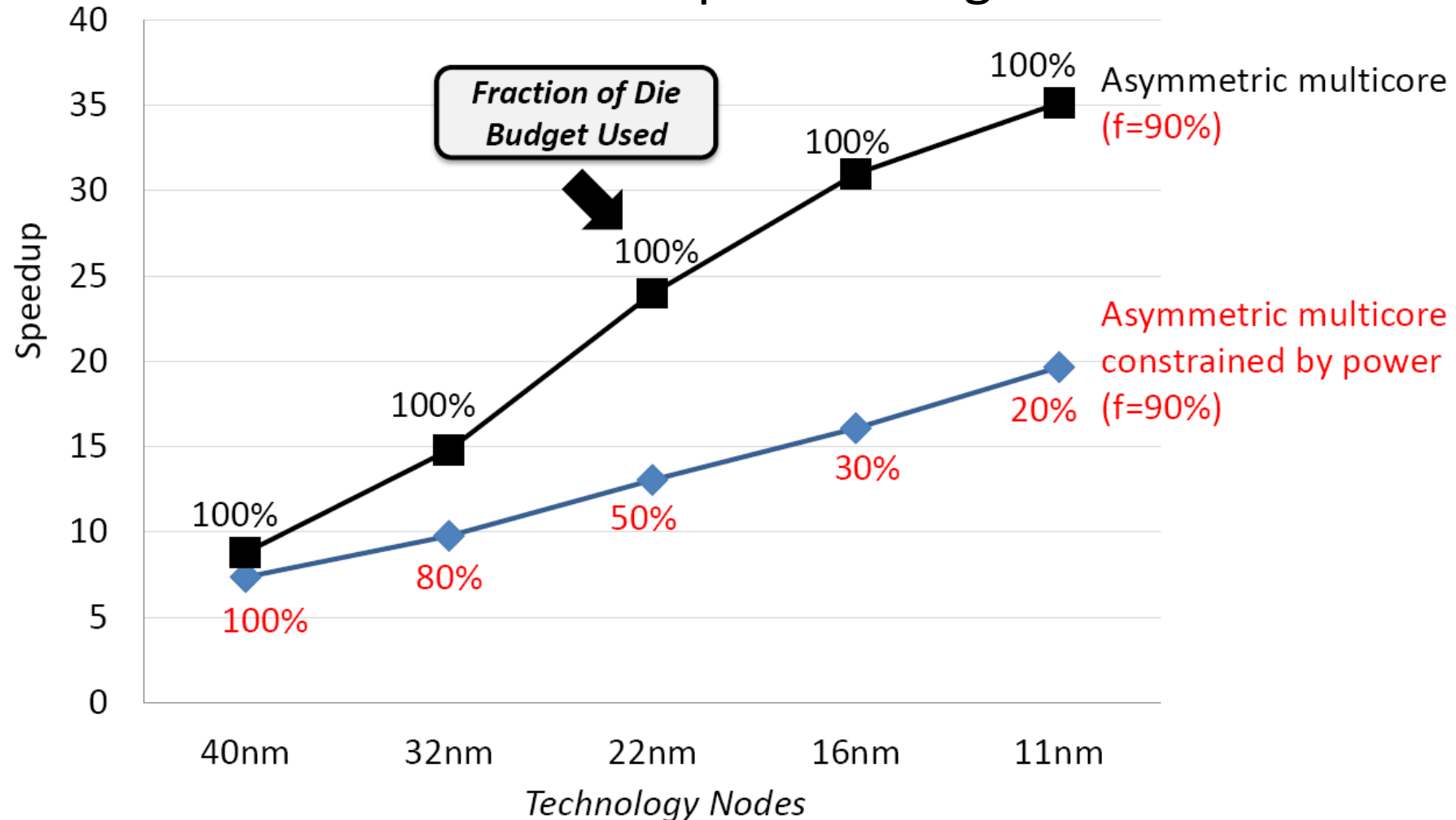
Heterogeneous Amdahl's Law – Matrix Multiply

Assume fixed die area $\sim 400\text{mm}^2$ and power budget 100W



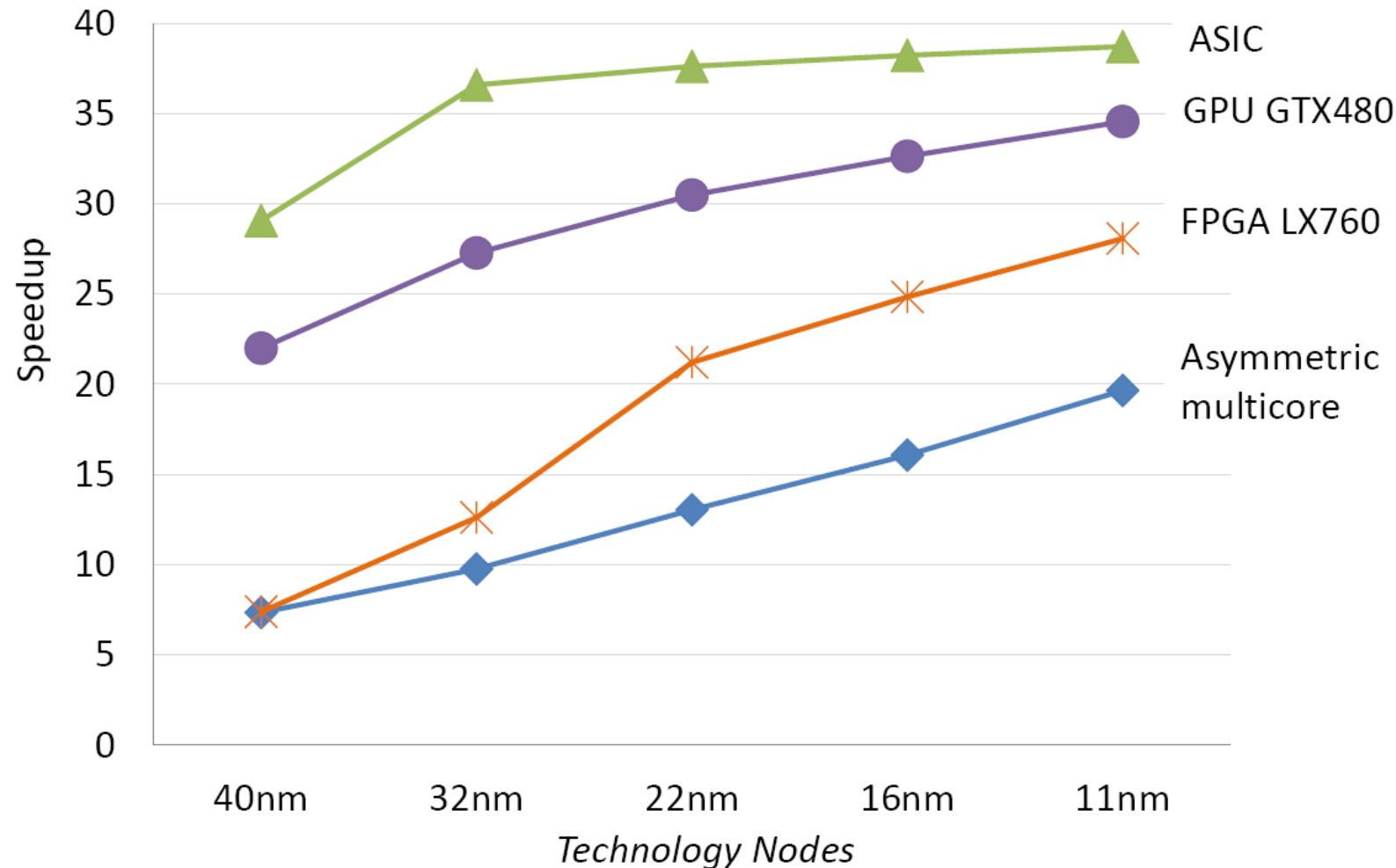
Heterogeneous Amdahl's Law – Matrix Multiply

Assume fixed die area $\sim 400\text{mm}^2$ and power budget 100W



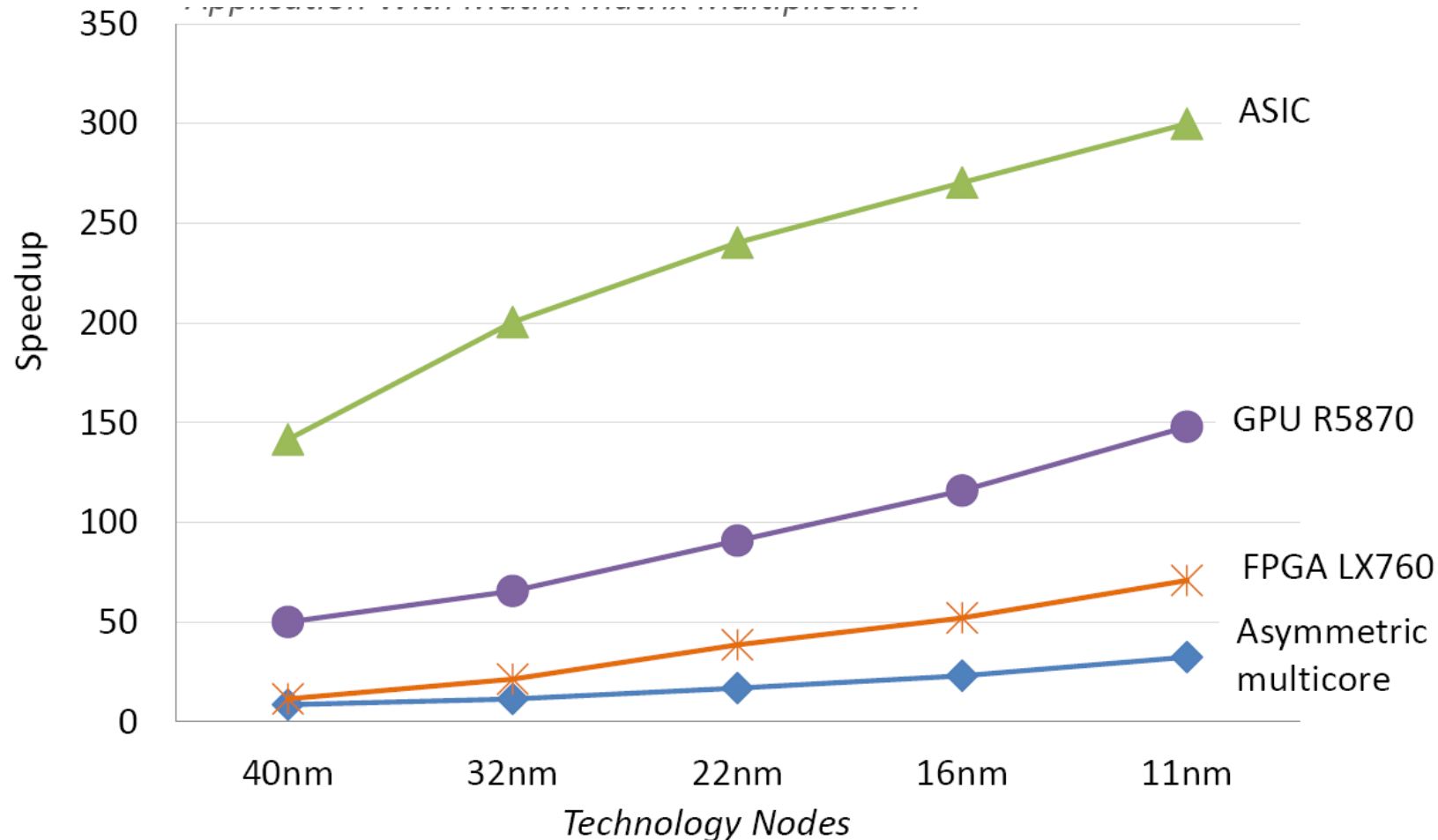
Heterogeneous Amdahl's Law – Matrix Multiply

Assume fixed die area $\sim 400\text{mm}^2$ and power budget 100W, $f=90\%$



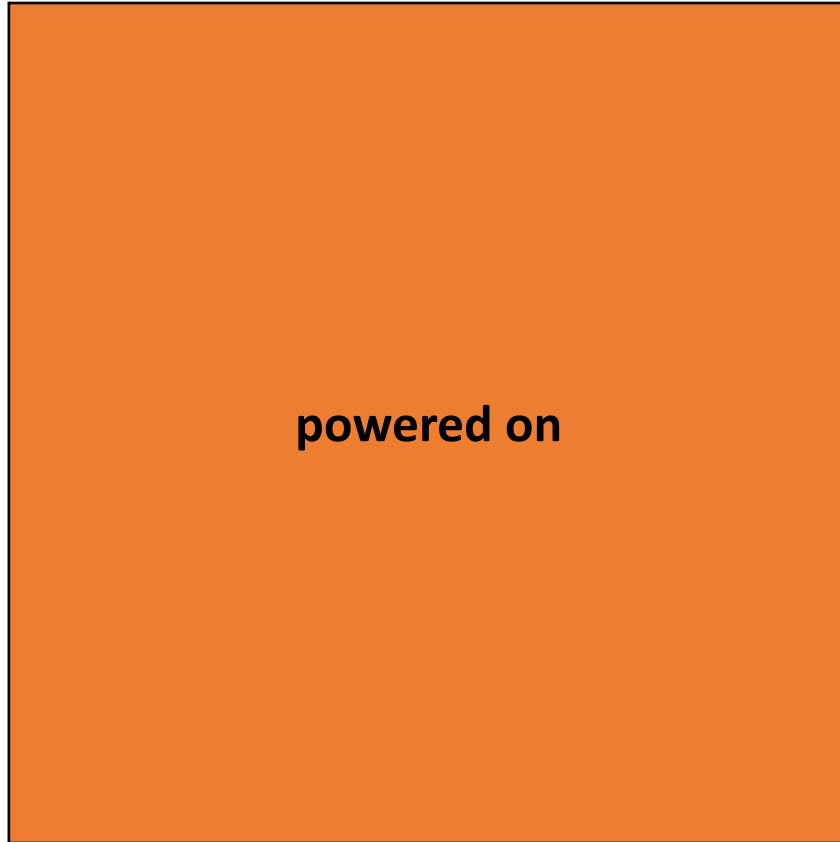
Heterogeneous Amdahl's Law – Matrix Multiply

Assume fixed die area $\sim 400\text{mm}^2$ and power budget 100W, $f=99\%$



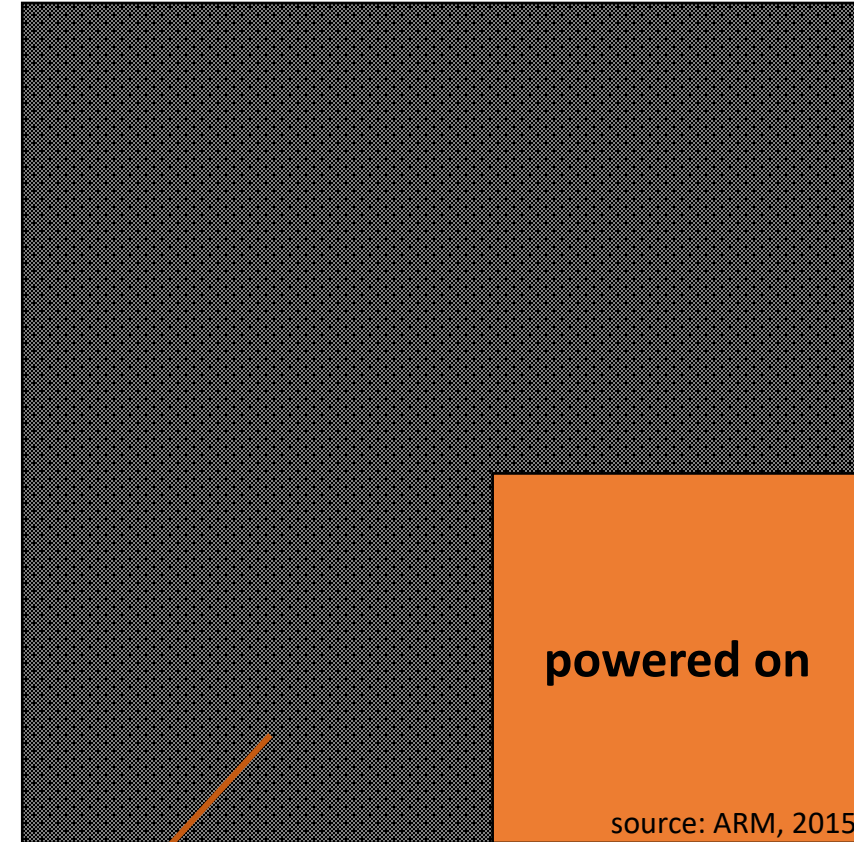
Specialization

Before



powered on

After (5 nm)



powered on

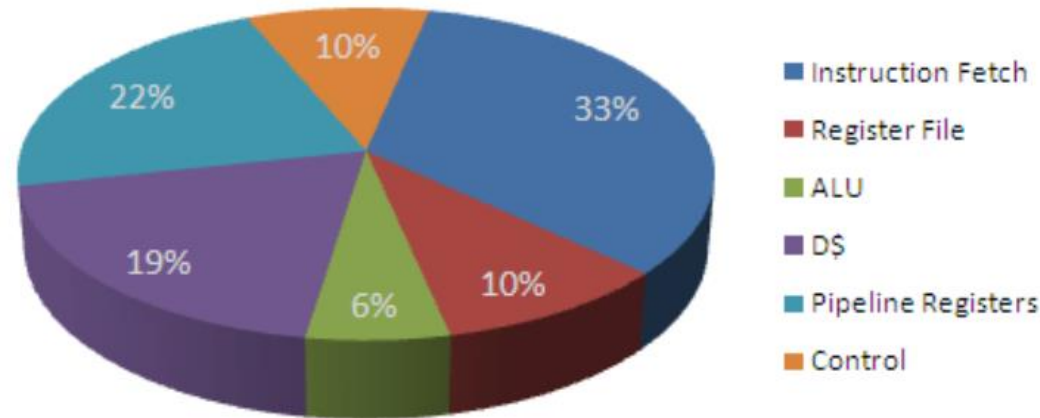
source: ARM, 2015

fill with specialized (heterogeneous) cores

Specialization

Sources of inefficiency in general-purpose processors

- e.g., H.264 video encoder pipeline
- Even if we assume everything but ALU is overhead, only 17x efficiency improvement



LogCA

M. S. B. Altaf and D. A. Wood, *LogCA: A High-Level Performance Model for Hardware Accelerators*, ISCA, 2017

How do we model performance of accelerators?

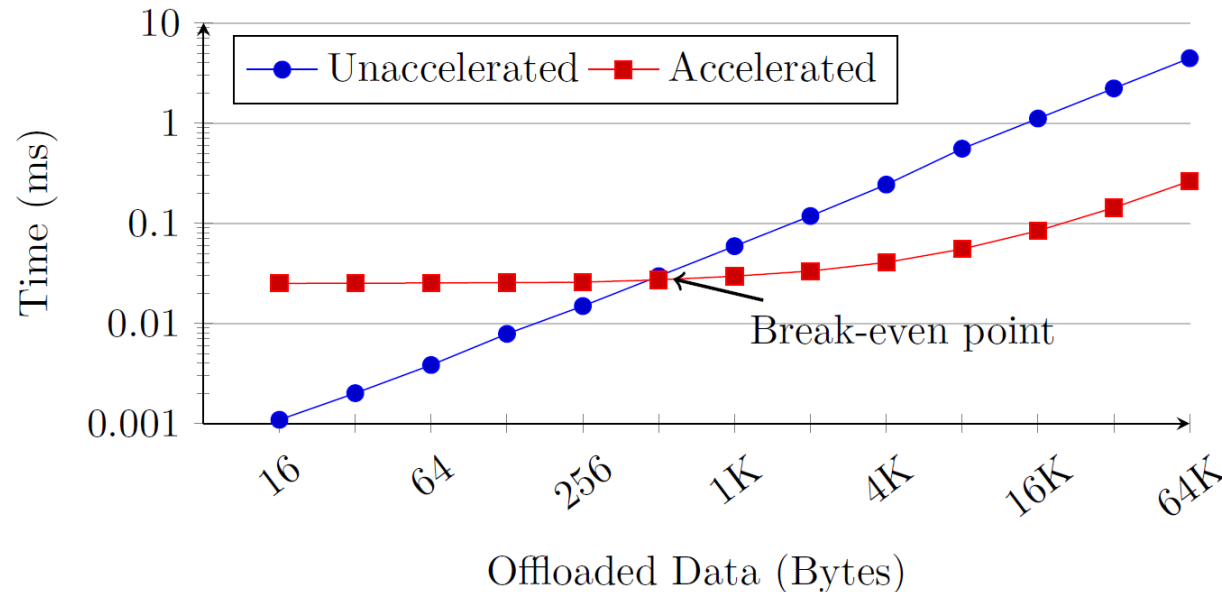
LogCA

Is acceleration always beneficial?

LogCA

Is acceleration always beneficial?

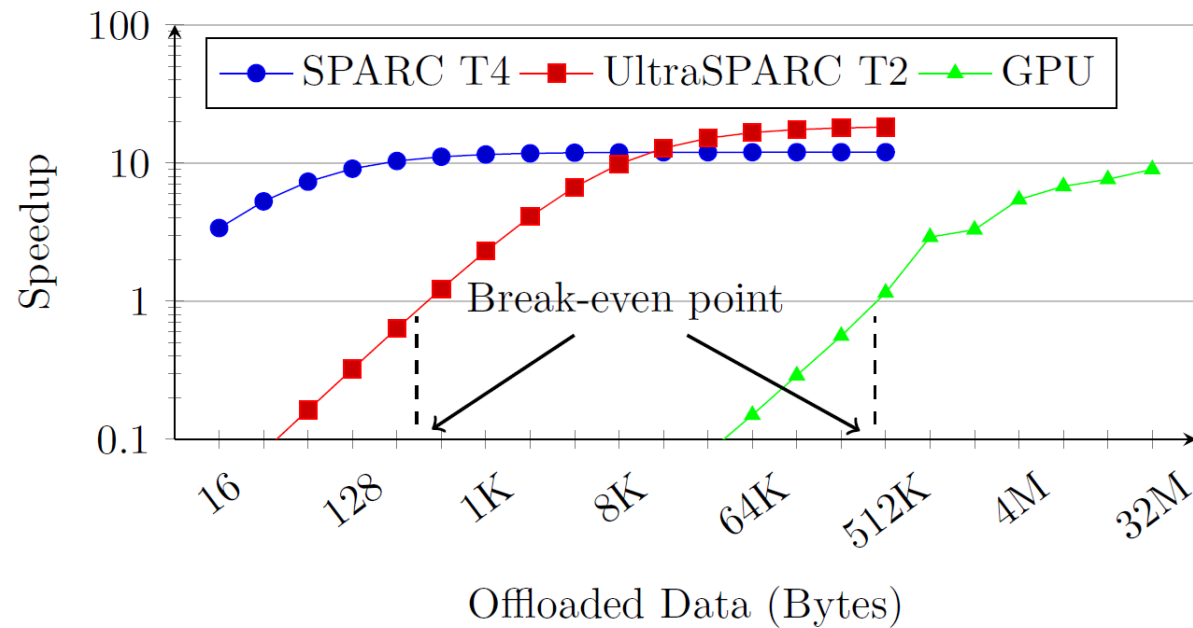
- Offloading a kernel to an accelerator incurs latency and overhead that depends on offloaded data
- e.g., AES encryption accelerator in UltraSPARC T2



LogCA

Is acceleration always beneficial?

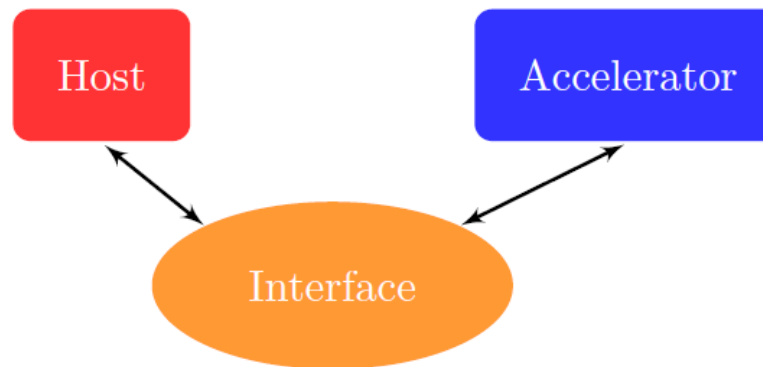
- e.g., AES encryption accelerator in SPARC T4, UltraSPARC T2 and GPU



LogCA

High-level model for accelerator performance

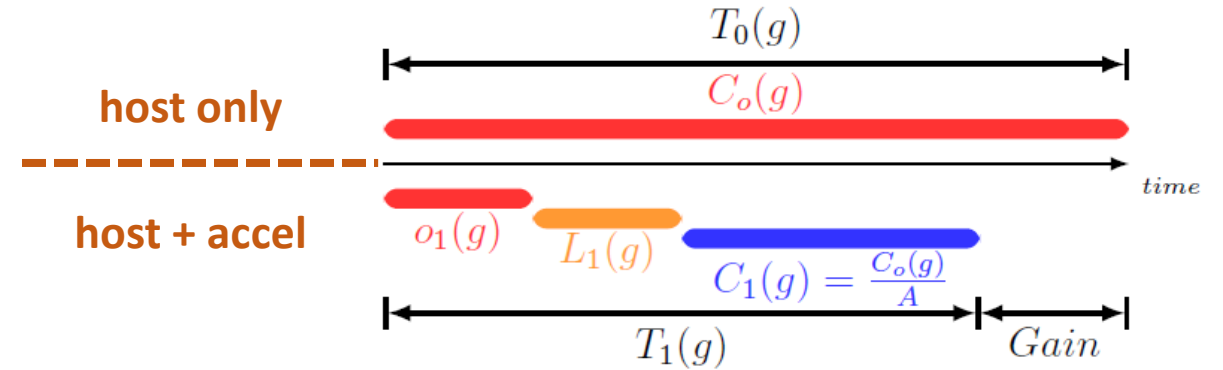
- **Host:** general-purpose processor
- **Accelerator:** hardware specifically designed to implement algorithm
- **Interface:** abstraction of interconnect and memory hierarchy



LogCA

Accelerator runtime consists of:

- Overhead of initialization at host
- Latency of sending offloaded data
- Computation on accelerator

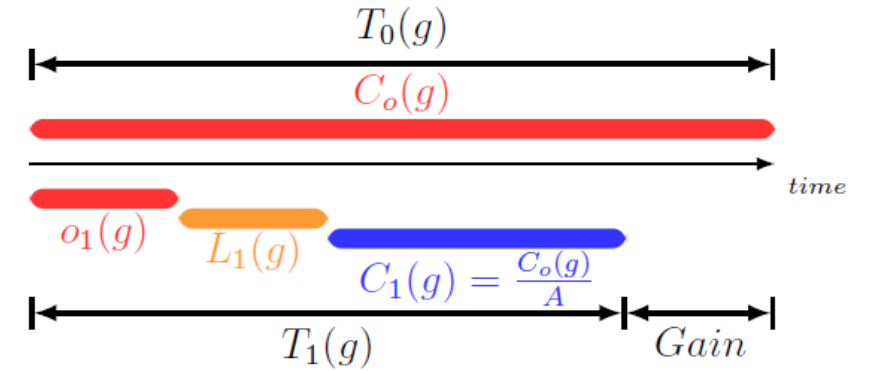


Parameter	Symbol	Description
Latency	L	Cycles to move data from the host to the accelerator across the interface, including the cycles data spends in the caches or memory
Overhead	o	Cycles the host spends in setting up the algorithm
Granularity	g	Size of the offloaded data
Computational Index	C	Cycles the host spends per byte of data
Acceleration	A	The peak speedup of an accelerator

LogCA

Overhead of initialization at host:

- Assume constant, $o_1(g) = 0$



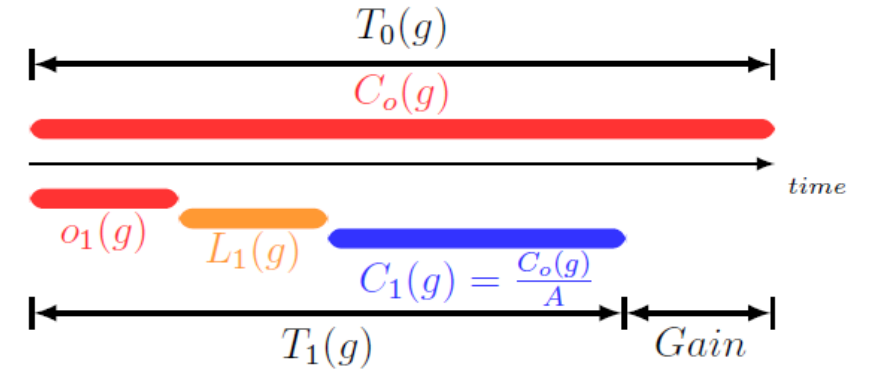
LogCA

Overhead of initialization at host:

- Assume constant, $o_1(g) = o$

Latency of sending offloaded data:

- Assume constant if pipelined, $L_1(g) = L$
- Or assume linear if not pipelined, $L_1(g) = L * g$



LogCA

Overhead of initialization at host:

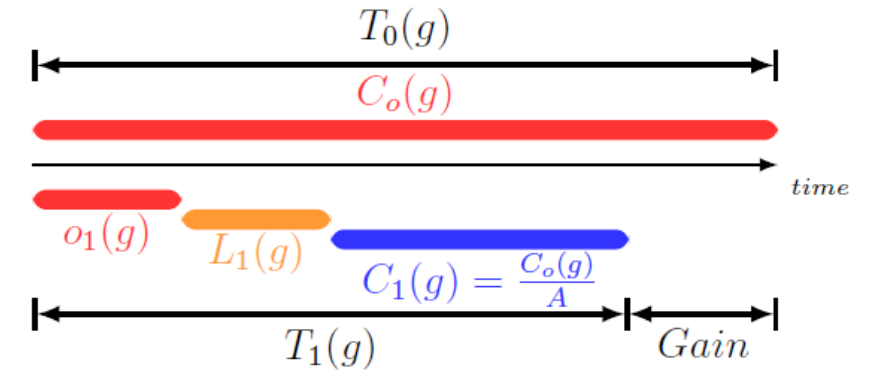
- Assume constant, $o_1(g) = o$

Latency of sending offloaded data:

- Assume constant if pipelined, $L_1(g) = L$
- Or assume linear if not pipelined, $L_1(g) = L * g$

Computation on accelerator:

- Assume linear speedup, $C_1(g) = C_0(g) / A$
- Assume computation on host has complexity $f(g)$ such that $C_0(g) = C * f(g)$
- Assume complexity is power function, $f(g) = O(g^\beta)$



LogCA

Overhead of initialization at host:

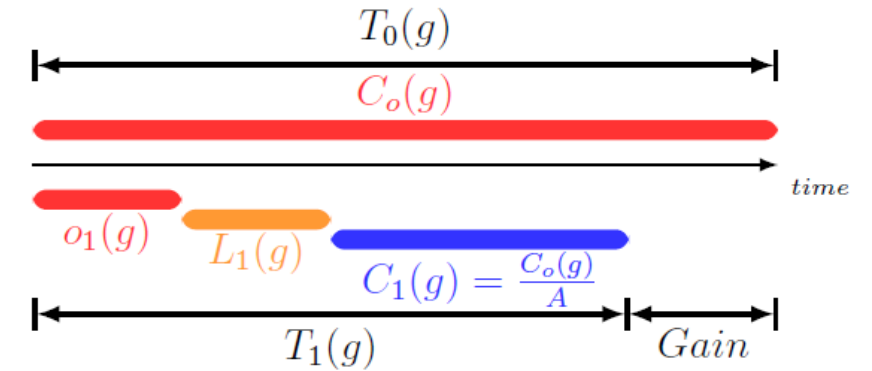
- Assume constant, $o_1(g) = o$

Latency of sending offloaded data:

- **Assume constant if pipelined, $L_1(g) = L$**
- Or assume linear if not pipelined, $L_1(g) = L * g$

Computation on accelerator:

- Assume linear speedup, $C_1(g) = C_0(g) / A$
- Assume computation on host has complexity $f(g)$ such that $C_0(g) = C * f(g)$
- Assume complexity is power function, $f(g) = O(g^\beta)$



$$Speedup(g) = \frac{C * f(g)}{o + L + \frac{C * f(g)}{A}} = \frac{C * g^\beta}{o + L + \frac{C * g^\beta}{A}}$$

LogCA

Overhead of initialization at host:

- Assume constant, $o_1(g) = o$

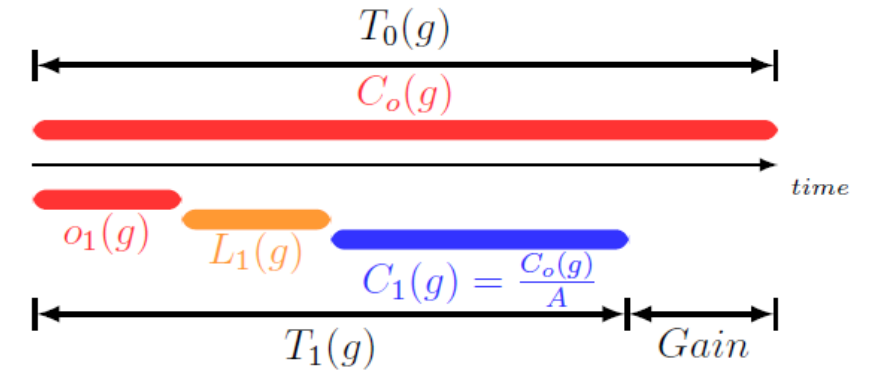
Latency of sending offloaded data:

- **Assume constant if pipelined, $L_1(g) = L$**
- Or assume linear if not pipelined, $L_1(g) = L * g$

Computation on accelerator:

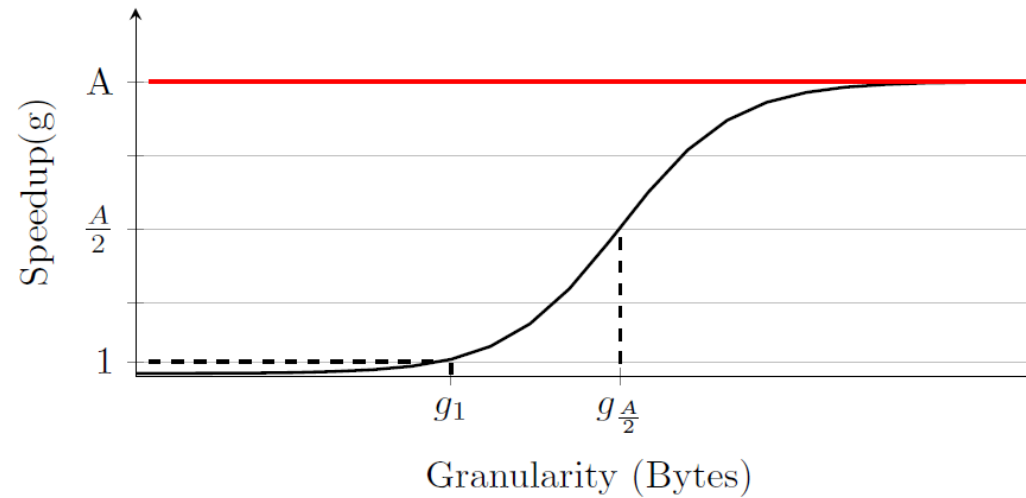
- Assume linear speedup, $C_1(g) = C_0(g) / A$
- Assume computation on host has complexity $f(g)$ such that $C_0(g) = C * f(g)$
- Assume complexity is power function, $f(g) = O(g^\beta)$

$$Speedup(g) = \frac{C * f(g)}{o + L + \frac{C * f(g)}{A}} = \frac{C * g^\beta}{o + L + \frac{C * g^\beta}{A}}$$



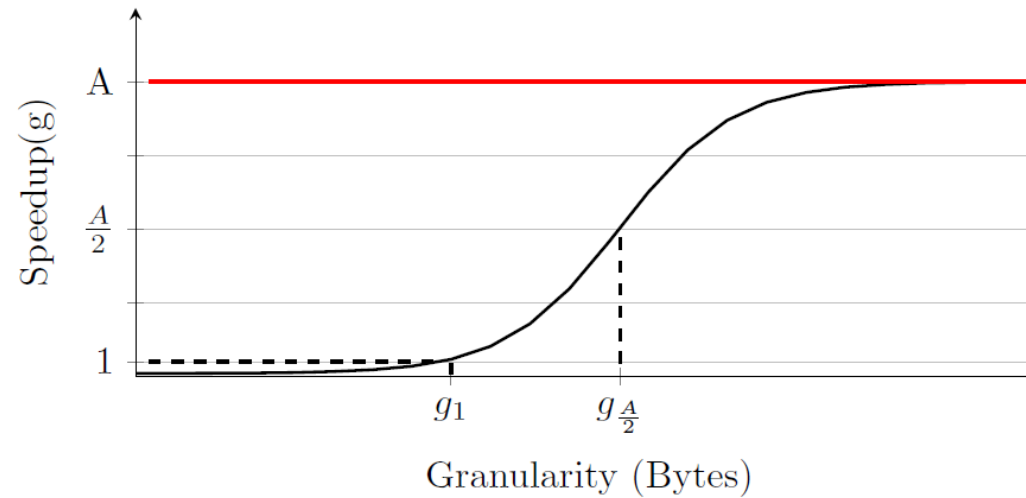
**linear speedup makes sense for parallelism,
but would accelerator necessarily have same complexity as host?**

LogCA – Implications



At large granularities, approach ideal speedup: $\lim_{g \rightarrow \infty} Speedup(g) = A$

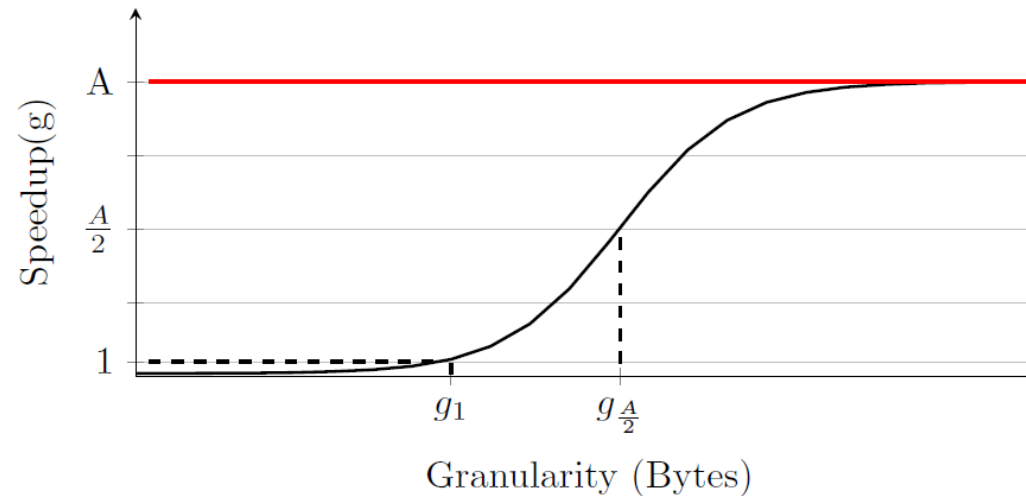
LogCA – Implications



At large granularities, approach ideal speedup: $\lim_{g \rightarrow \infty} \text{Speedup}(g) = A$

At small granularities, overheads dominate: $\lim_{g \rightarrow 0} \text{Speedup}(g) \simeq \frac{C}{o + L + \frac{C}{A}} < \frac{C}{o + L}$

LogCA – Implications

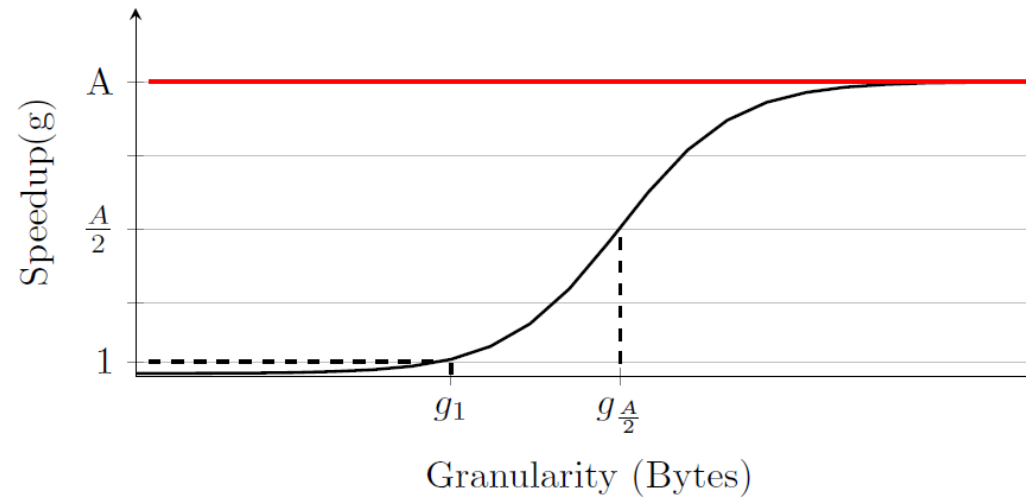


At large granularities, approach ideal speedup: $\lim_{g \rightarrow \infty} \text{Speedup}(g) = A$

If the work being accelerated (C) exceeds the cost of accelerating it ($o+L$), upper bound speedup > 1

At small granularities, overheads dominate: $\lim_{g \rightarrow 0} \text{Speedup}(g) \simeq \frac{C}{o + L + \frac{C}{A}} < \frac{C}{o + L}$

LogCA – Implications



When A is very fast, we effectively pay the cost of $o+L$ to speed up C amount of work

At large granularities, approach ideal speedup: $\lim_{g \rightarrow \infty} \text{Speedup}(g) = A$

If the work being accelerated (C) exceeds the cost of accelerating it ($o+L$), upper bound speedup > 1

At small granularities, overheads dominate: $\lim_{g \rightarrow 0} \text{Speedup}(g) \simeq \frac{C}{o+L+\frac{C}{A}} < \frac{C}{o+L}$

LogCA – Granularity Dependent Latency

Overhead of initialization at host:

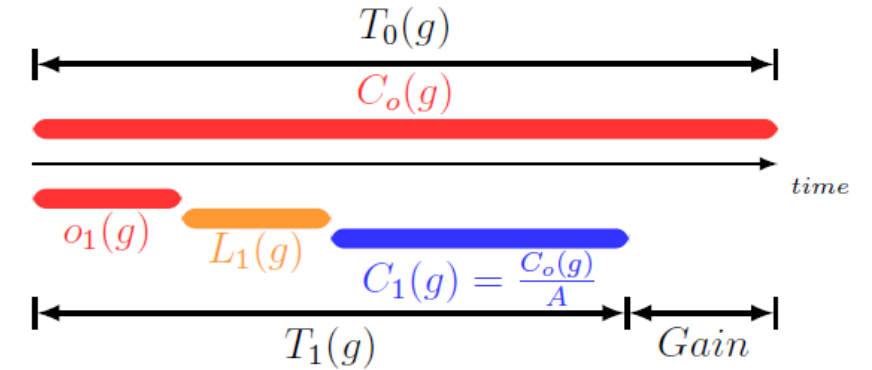
- Assume constant, $o_1(g) = o$

Latency of sending offloaded data:

- Assume constant if pipelined, $L_1(g) = L$
- **Or assume linear if not pipelined, $L_1(g) = L * g$**

Computation on accelerator:

- Assume linear speedup, $C_1(g) = C_0(g) / A$
- Assume computation on host has complexity $f(g)$ such that $C_0(g) = C * f(g)$
- Assume complexity is power function, $f(g) = O(g^\beta)$



$$Speedup(g) = \frac{C * g^\beta}{o + L * g + \frac{C * g^\beta}{A}}$$

LogCA – Granularity Dependent Latency

At large granularities, upper bound is dictated by computation-communication ratio (C/L):

- Upper bound is independent of A

$$\lim_{g \rightarrow \infty} \text{Speedup}(g) = \left(\frac{A}{\frac{A}{C * g^\beta} * (L * g) + 1} \right) < \frac{C}{L} * g^{\beta-1}$$



LogCA – Granularity Dependent Latency

At large granularities, upper bound is dictated by computation-communication ratio (C/L):

- Upper bound is independent of A

$$\lim_{g \rightarrow \infty} Speedup(g) = \left(\frac{A}{\frac{A}{C * g^\beta} * (L * g) + 1} \right) < \frac{C}{L} * g^{\beta-1}$$

When A infinite, each offloaded byte saves C cycles at the cost of L cycles



LogCA – Granularity Dependent Latency

At large granularities, upper bound is dictated by computation-communication ratio (C/L):

- Upper bound is independent of A
- Sub-linear algorithms ($\beta < 1$) see diminishing returns with larger granularities

$$\lim_{g \rightarrow \infty} \text{Speedup}(g) = \left(\frac{A}{\frac{A}{C * g^\beta} * (L * g) + 1} \right) < \frac{C}{L} * g^{\beta-1}$$

Why?

LogCA – Granularity Dependent Latency

At large granularities, upper bound is dictated by computation-communication ratio (C/L):

- Upper bound is independent of A
- Sub-linear algorithms ($\beta < 1$) see diminishing returns with larger granularities

$$\lim_{g \rightarrow \infty} \text{Speedup}(g) = \left(\frac{A}{\frac{A}{C * g^\beta} * (L * g) + 1} \right) < \frac{C}{L} * g^{\beta-1}$$

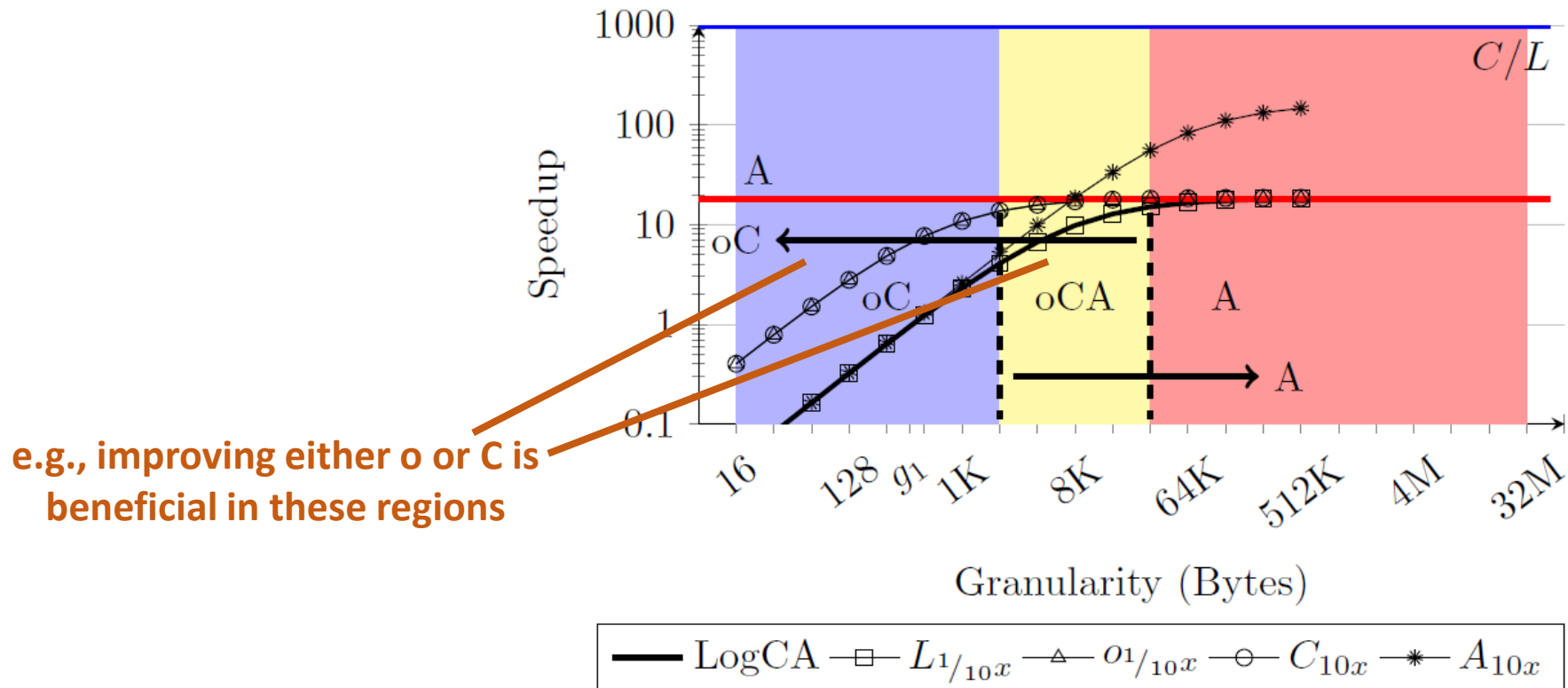
Why? Because model assumes that communication latency is a linear function of granularity ($L * g$), which eventually makes the acceleration of sub-linear computations less effective.



LogCA – Identifying Bottlenecks

e.g., AES accelerator on UltraSPARC T2

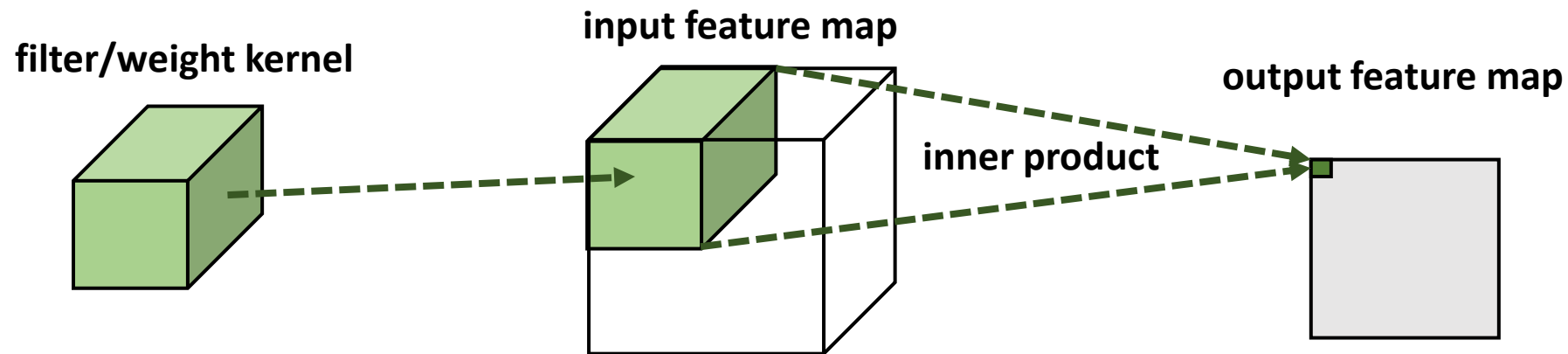
- Use model to identify optimization regions where improving each parameter by 10x provides at least 20% total speedup



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

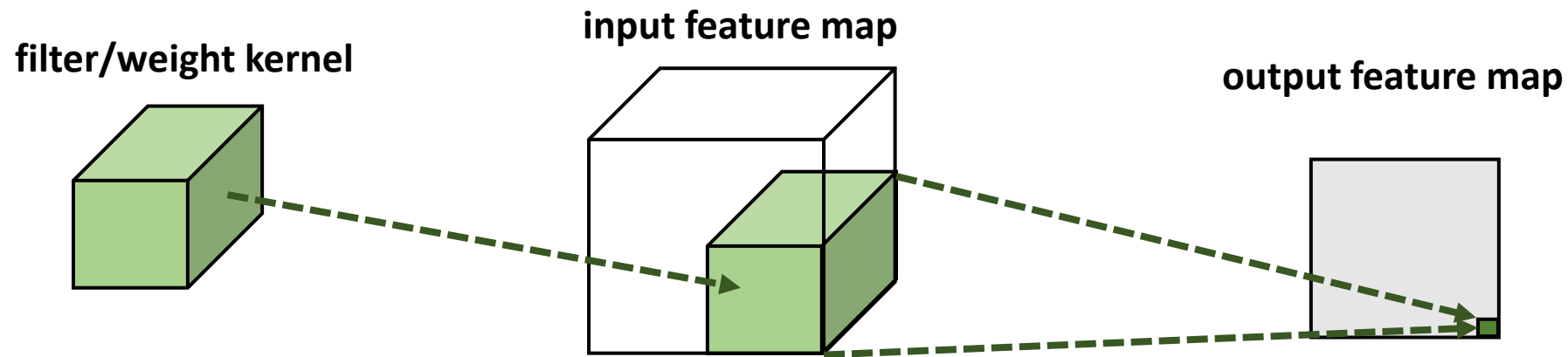
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

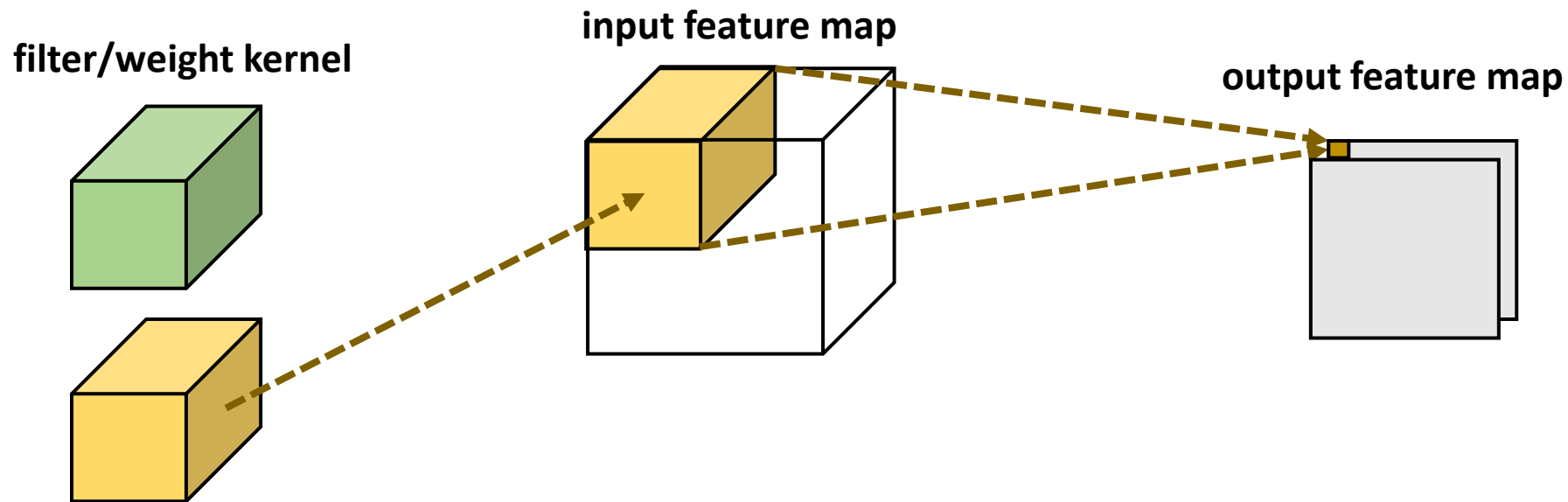
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

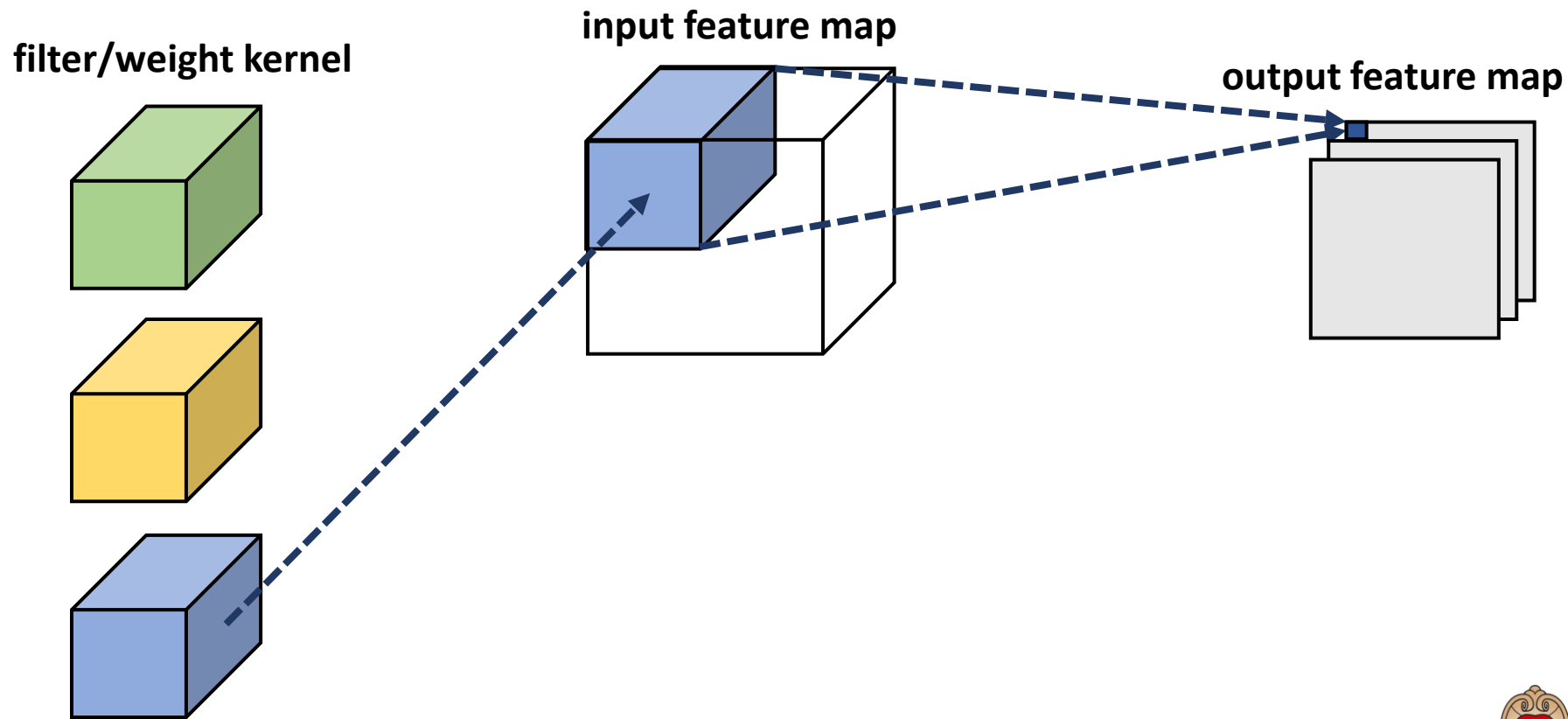
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

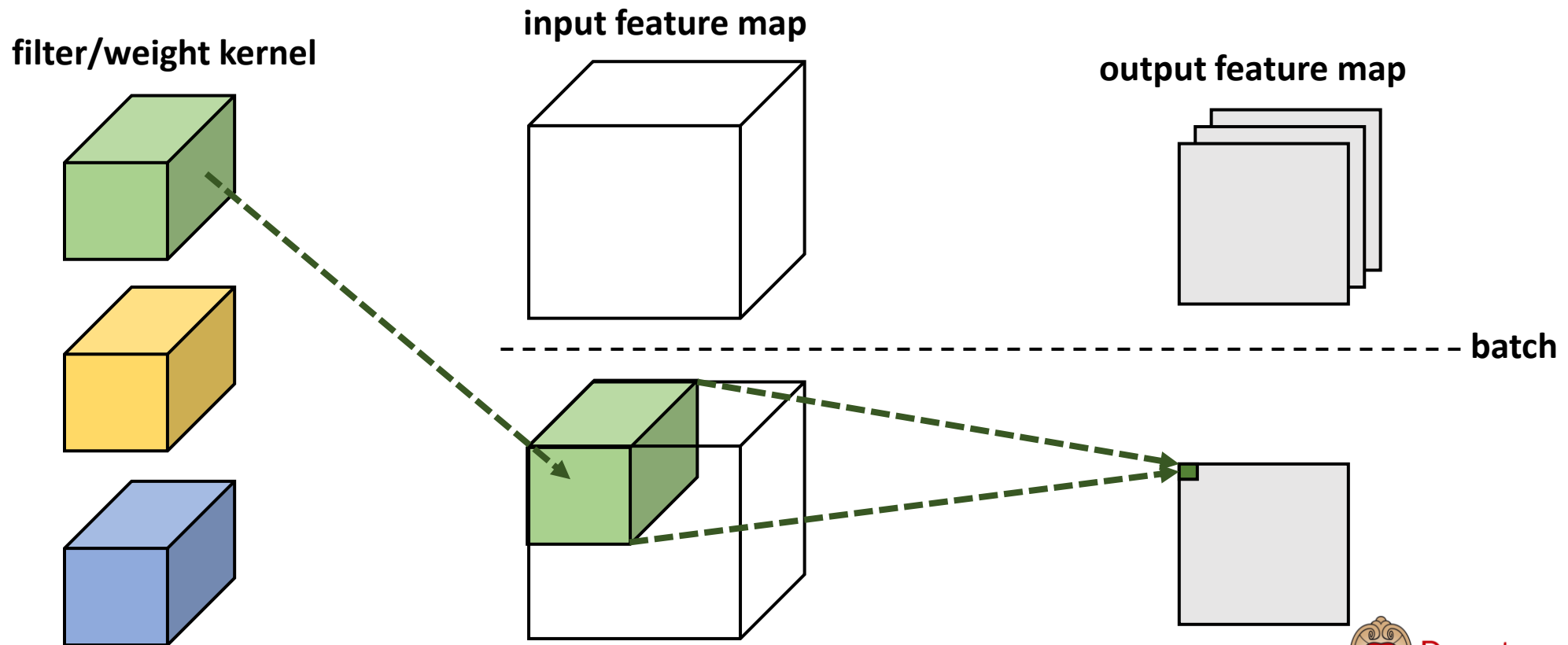
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

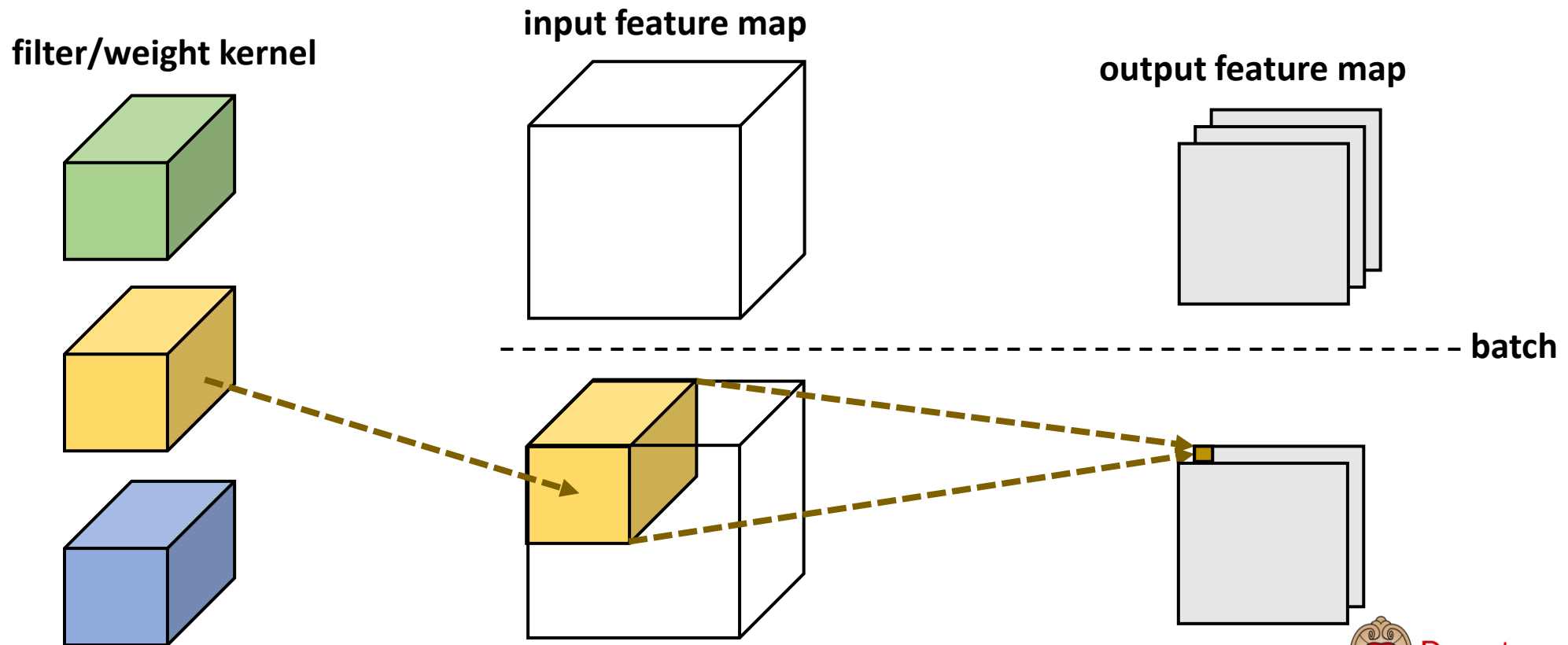
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

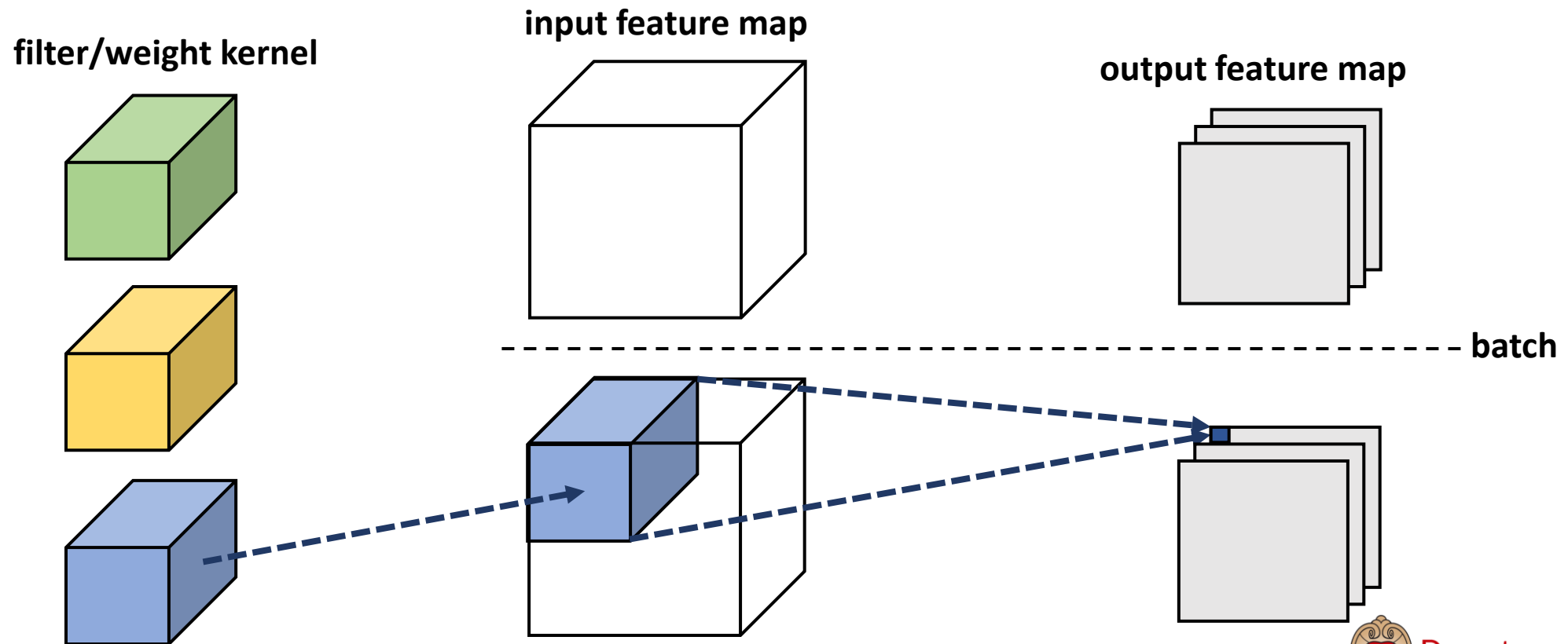
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators

Goal is to provide architecture for convolution and other matrix algebra

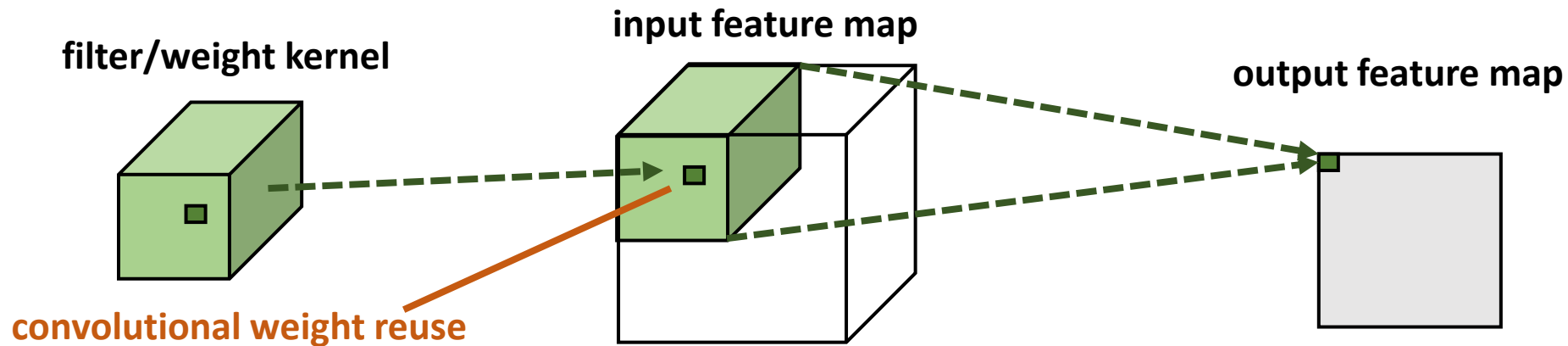
- Fully-connected layers: filter size = input feature map size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

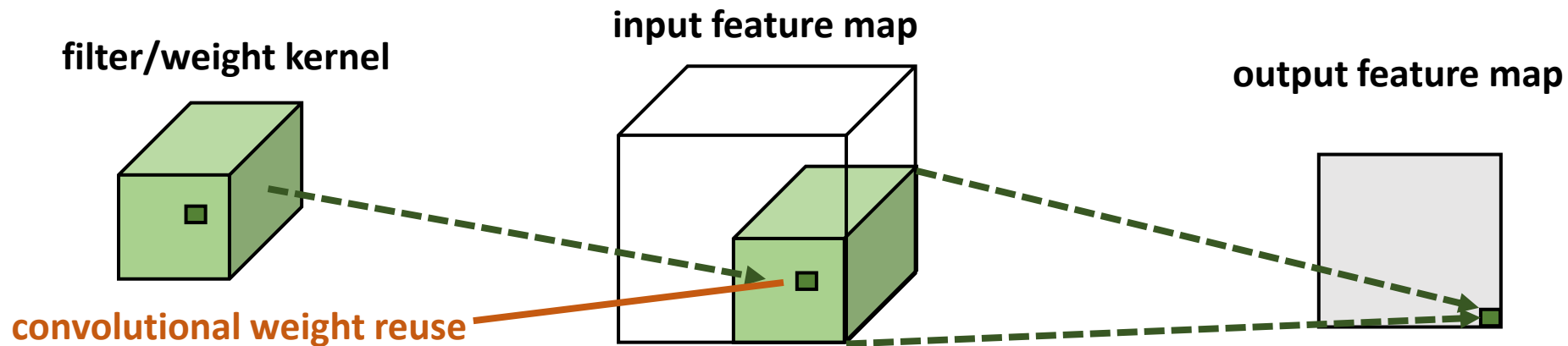
- **Convolutional Weight Reuse:** Proportional to output feature map size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

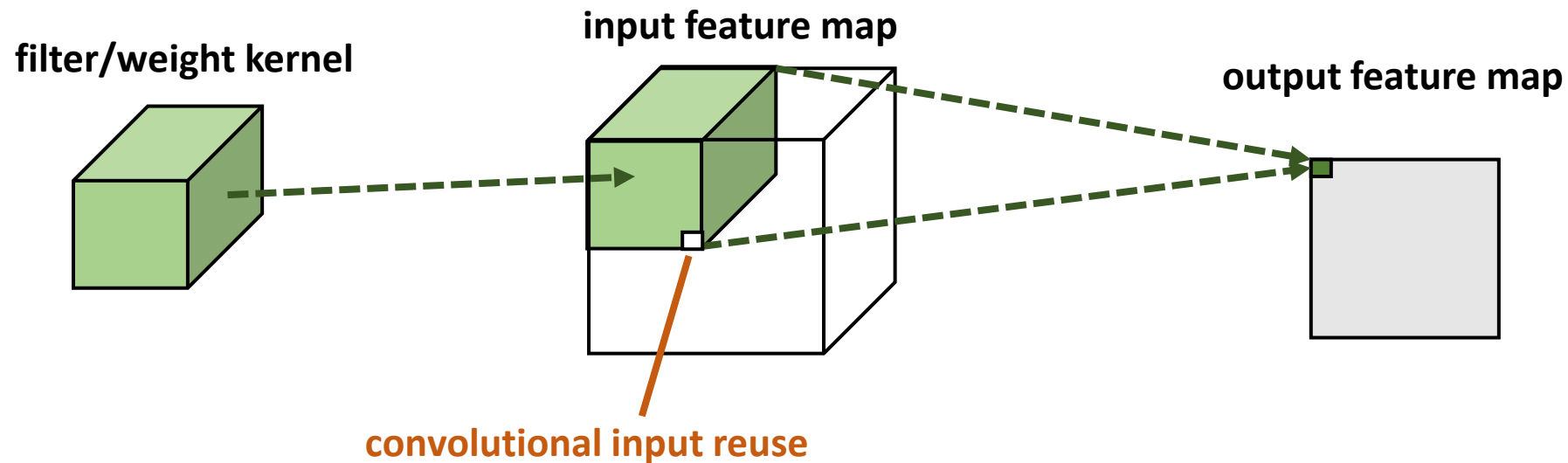
- **Convolutional Weight Reuse:** Proportional to output feature map size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

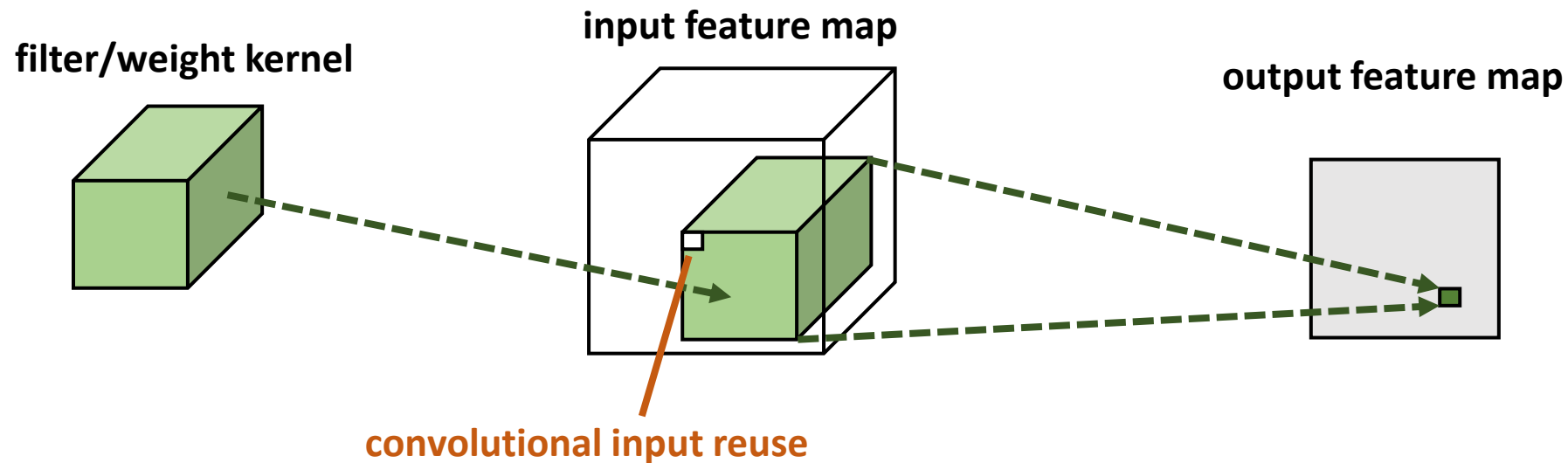
- **Convolutional Input Reuse:** Proportional to filter size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

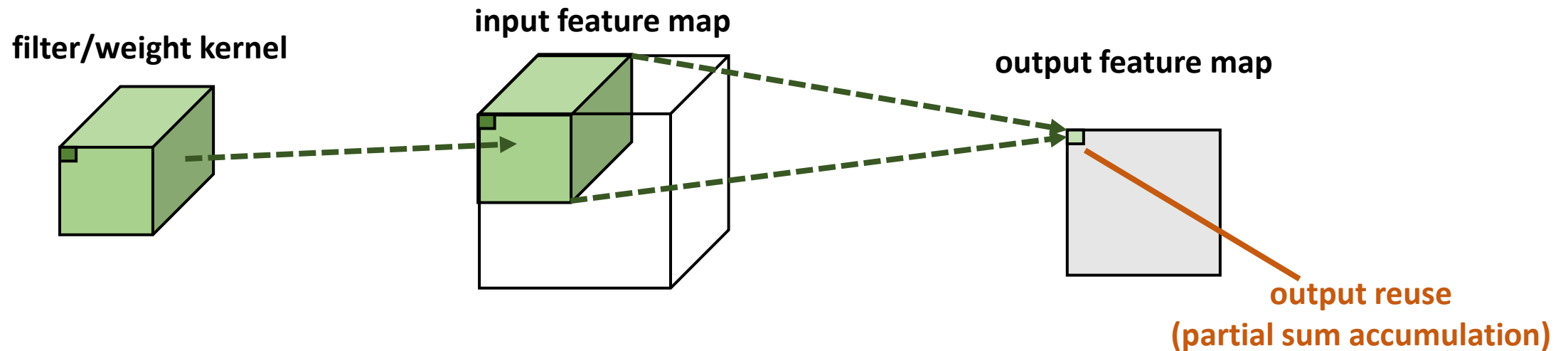
- **Convolutional Input Reuse:** Proportional to filter size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

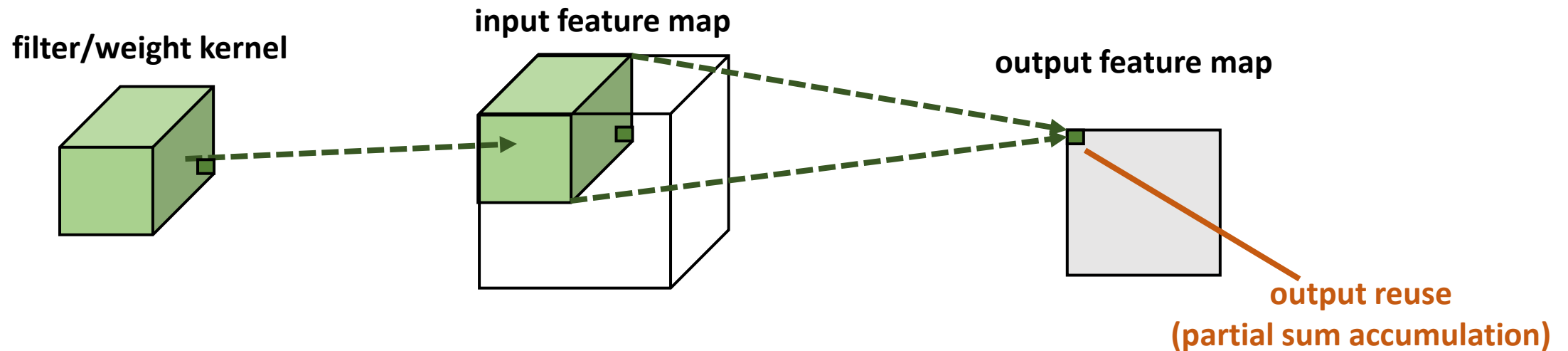
- **Output Reuse:** Proportional to filter size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

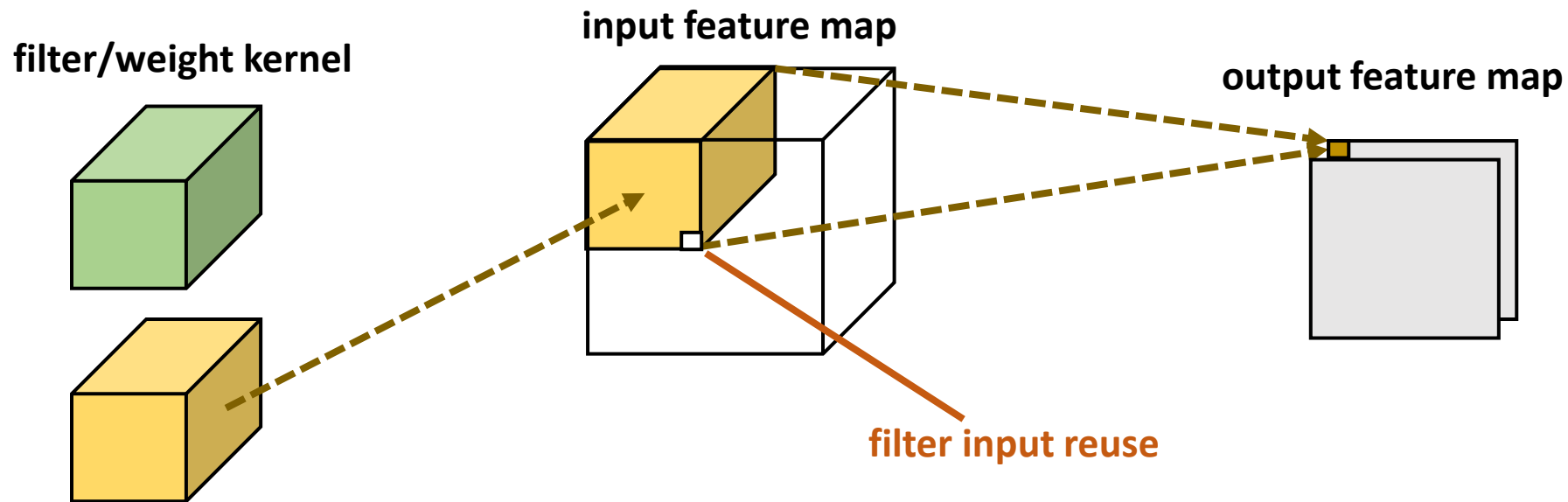
- **Output Reuse:** Proportional to filter size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

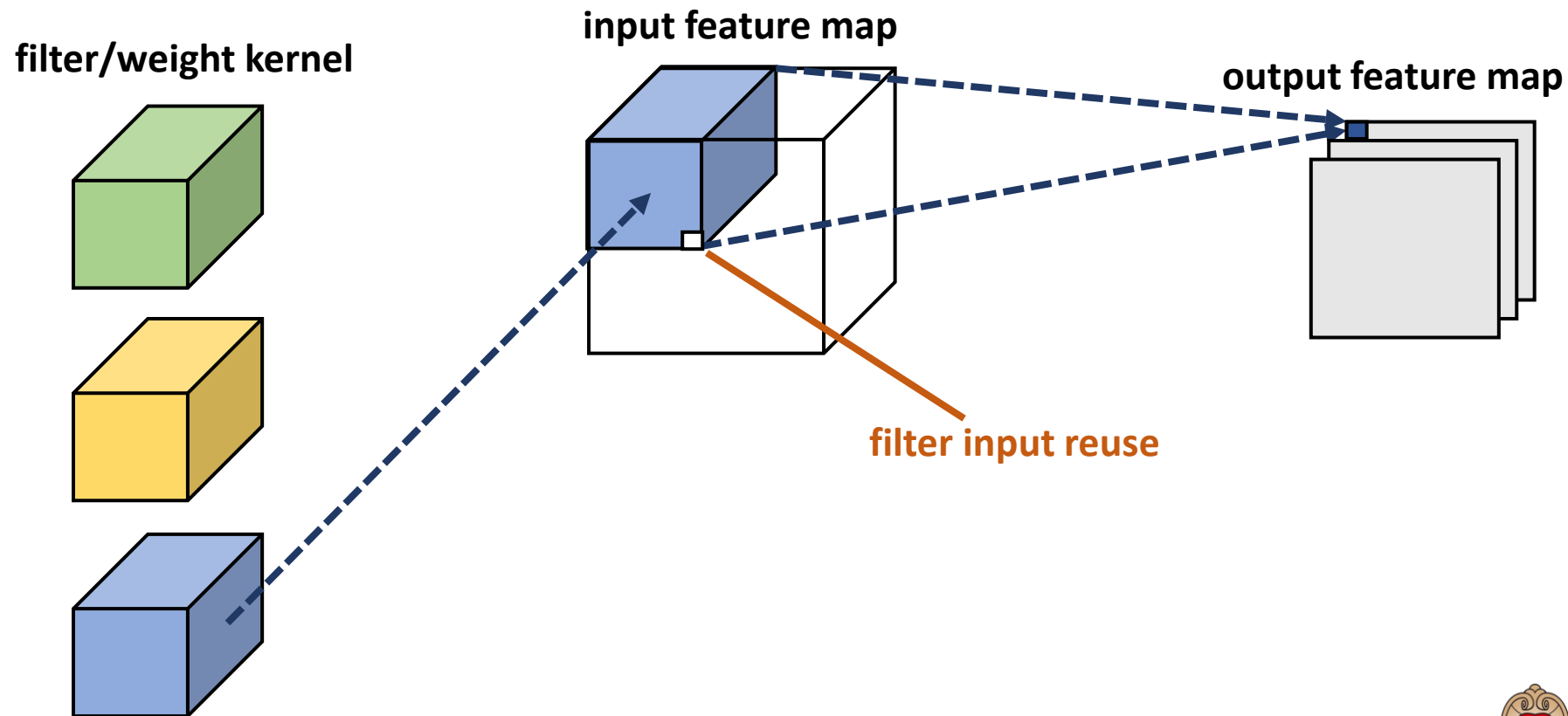
- **Filter Input Reuse:** Proportional to number of filters



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

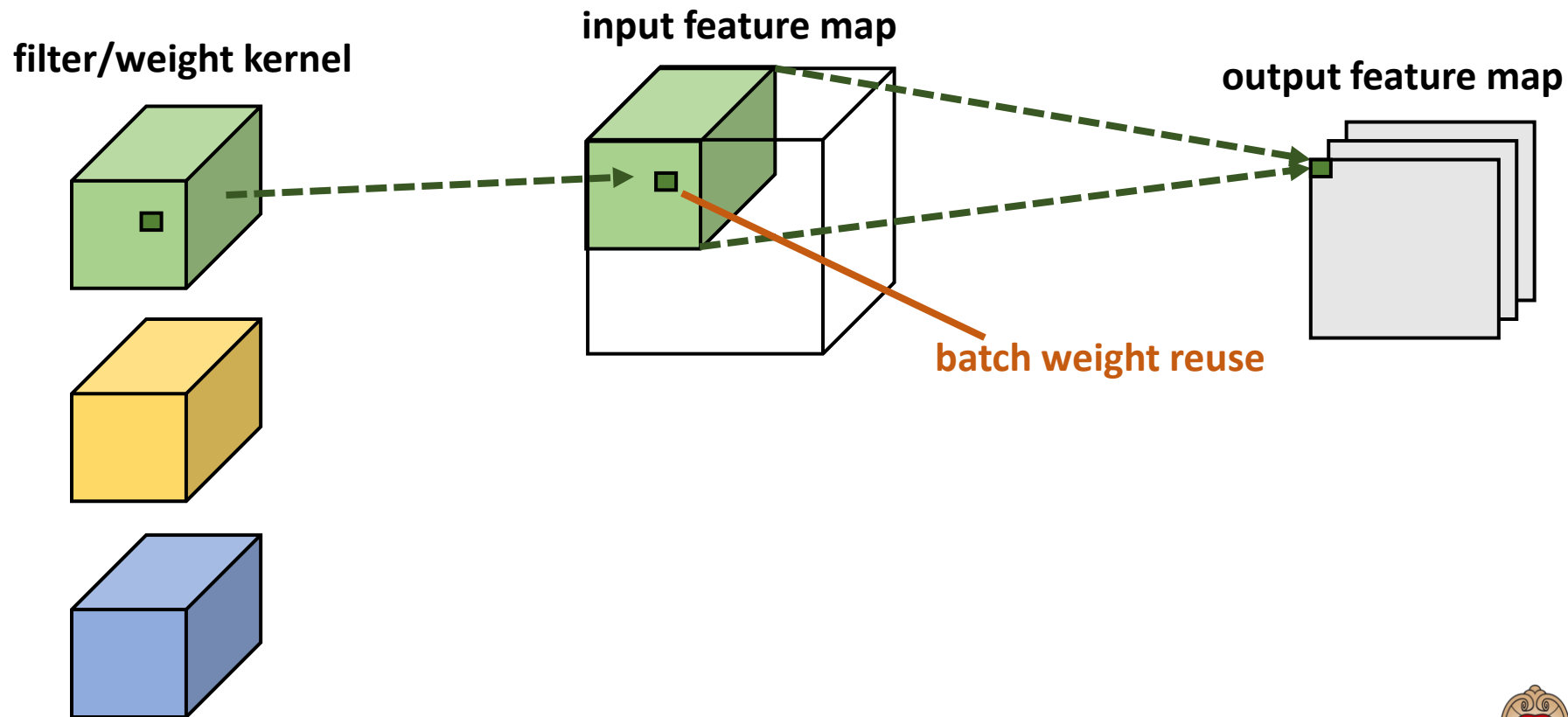
- **Filter Input Reuse:** Proportional to number of filters



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

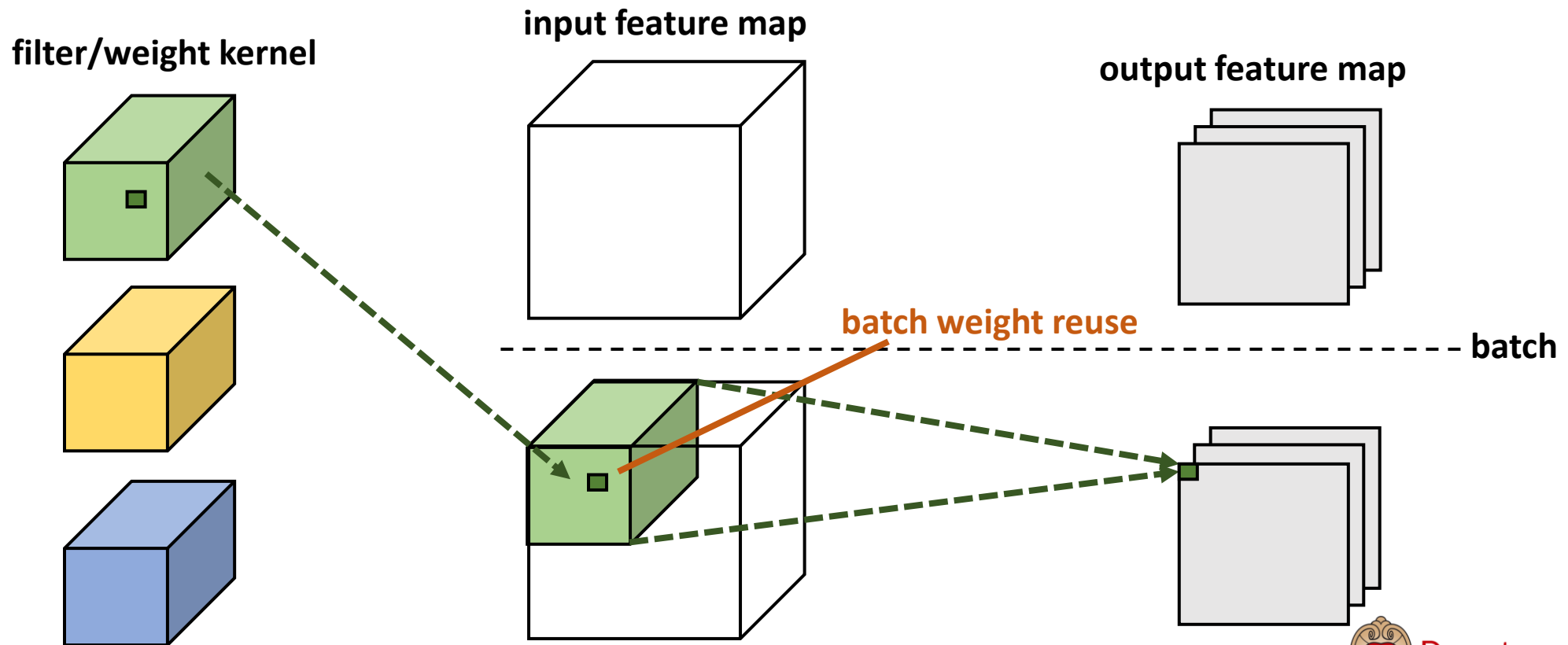
- **Batch Weight Reuse:** Proportional to batch size



Neural Network Accelerators – Reuse

Key types of data reuse in NN processing

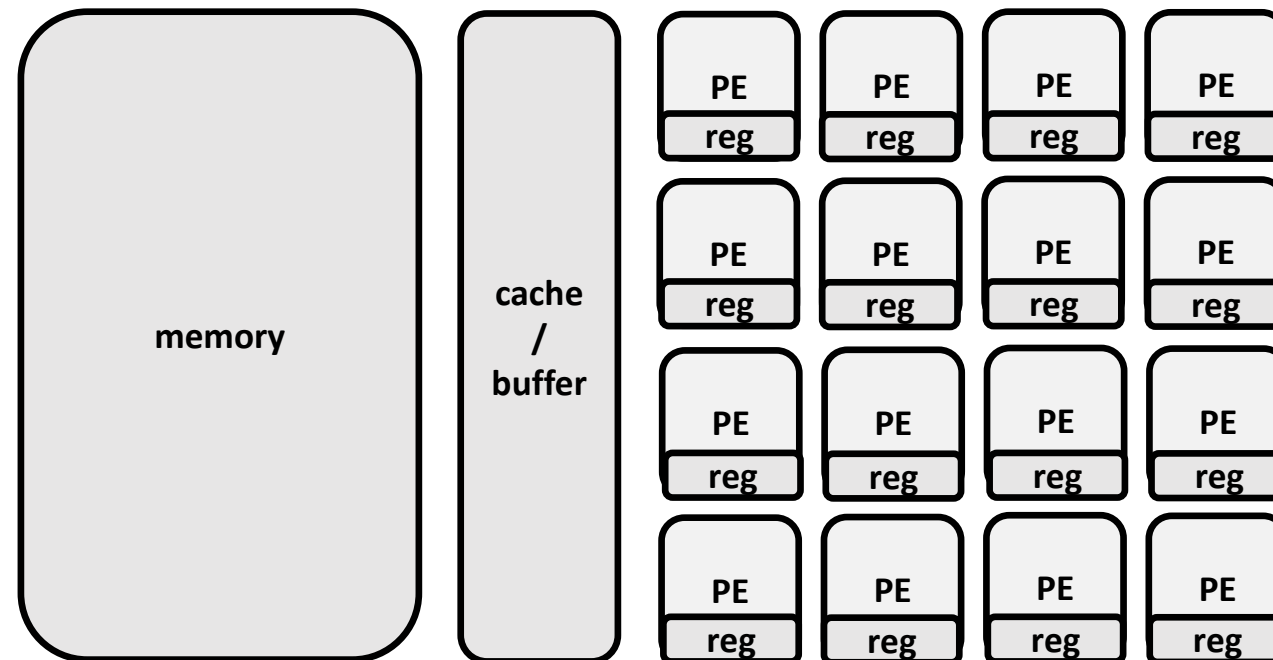
- **Batch Weight Reuse:** Proportional to batch size



Neural Network Accelerators – Architecture

General architecture of NN acceleration:

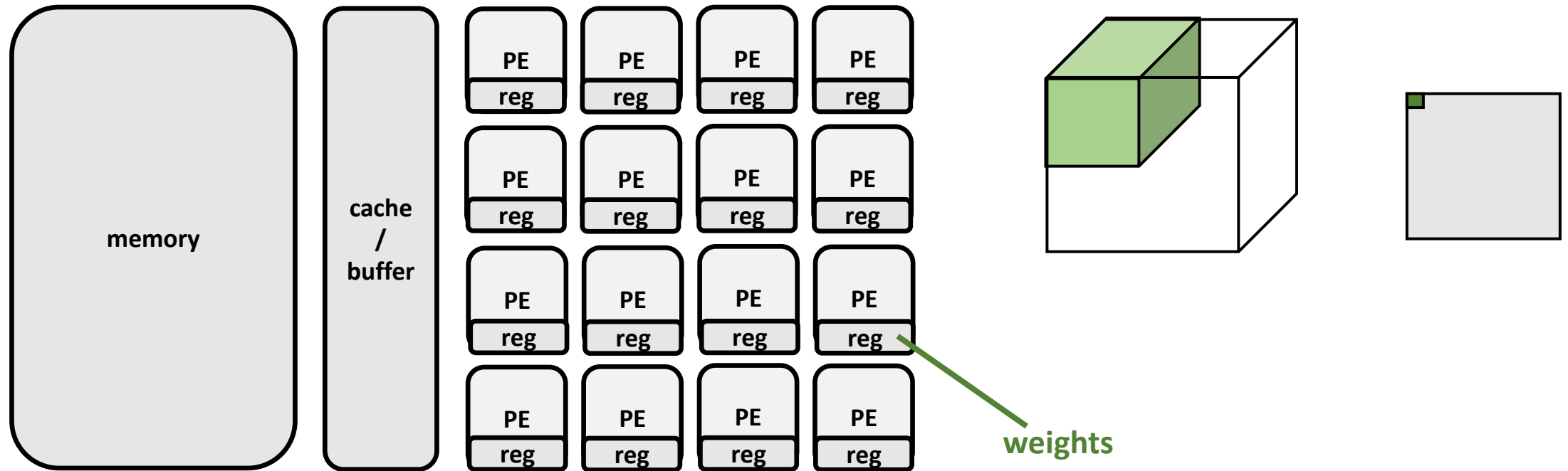
- Processing elements (PEs) with support for multiply-accumulate (MAC) and registers
- Software- or hardware-managed cache or scratchpad buffer
- Programmable network-on-chip to support *dataflow* of current NN layer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

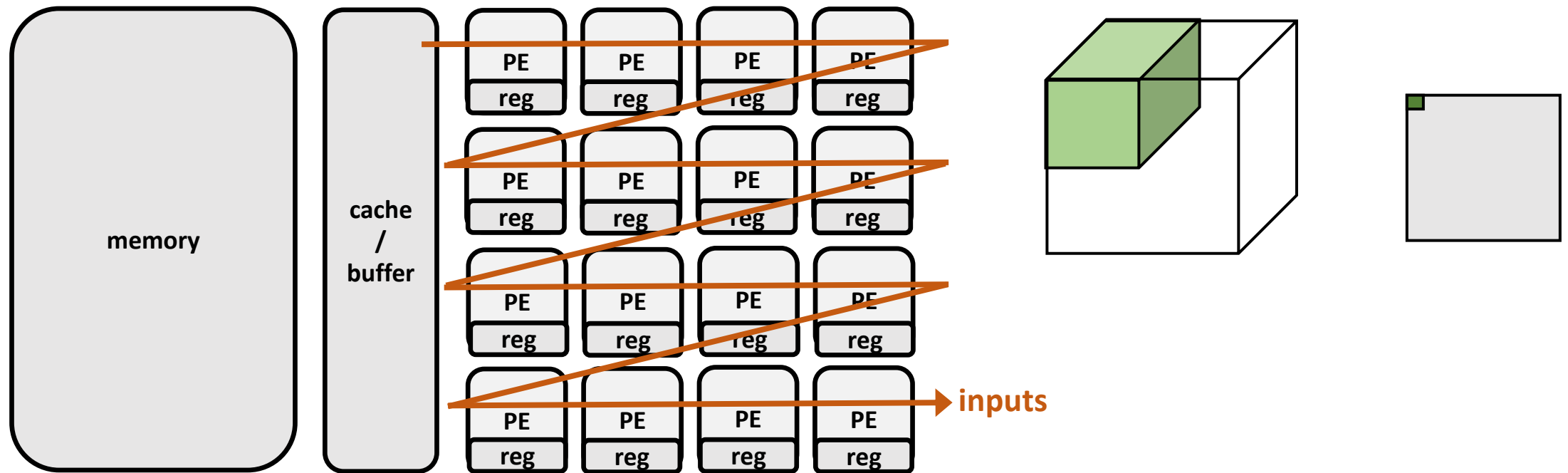
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

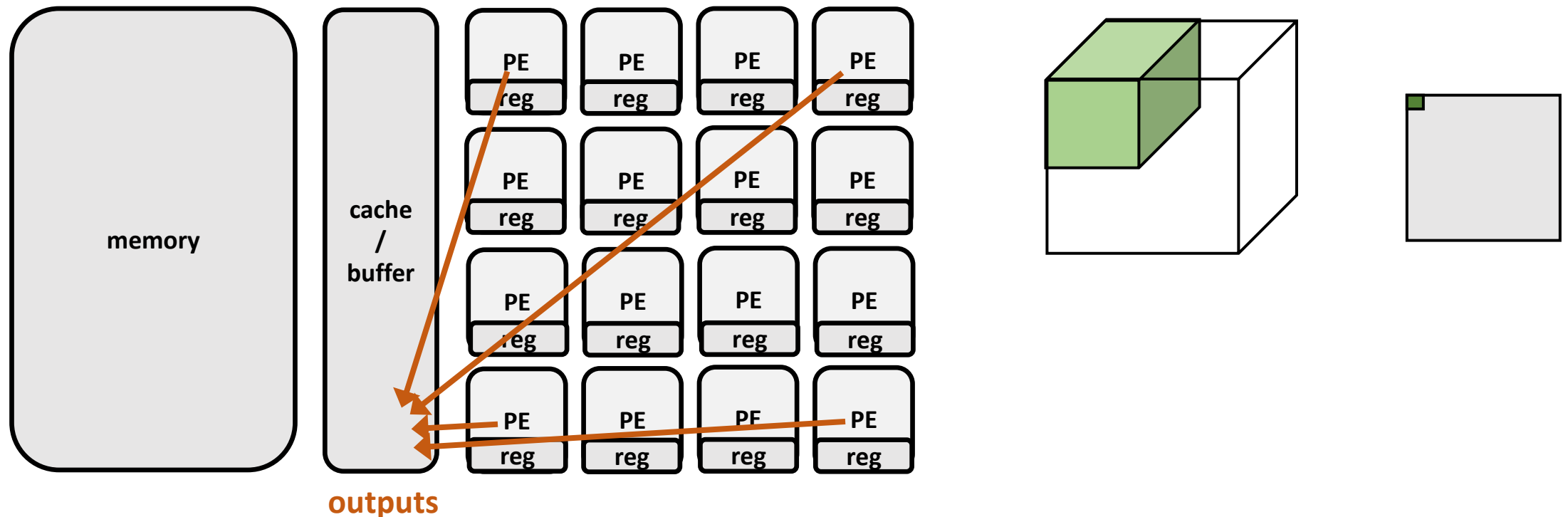
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

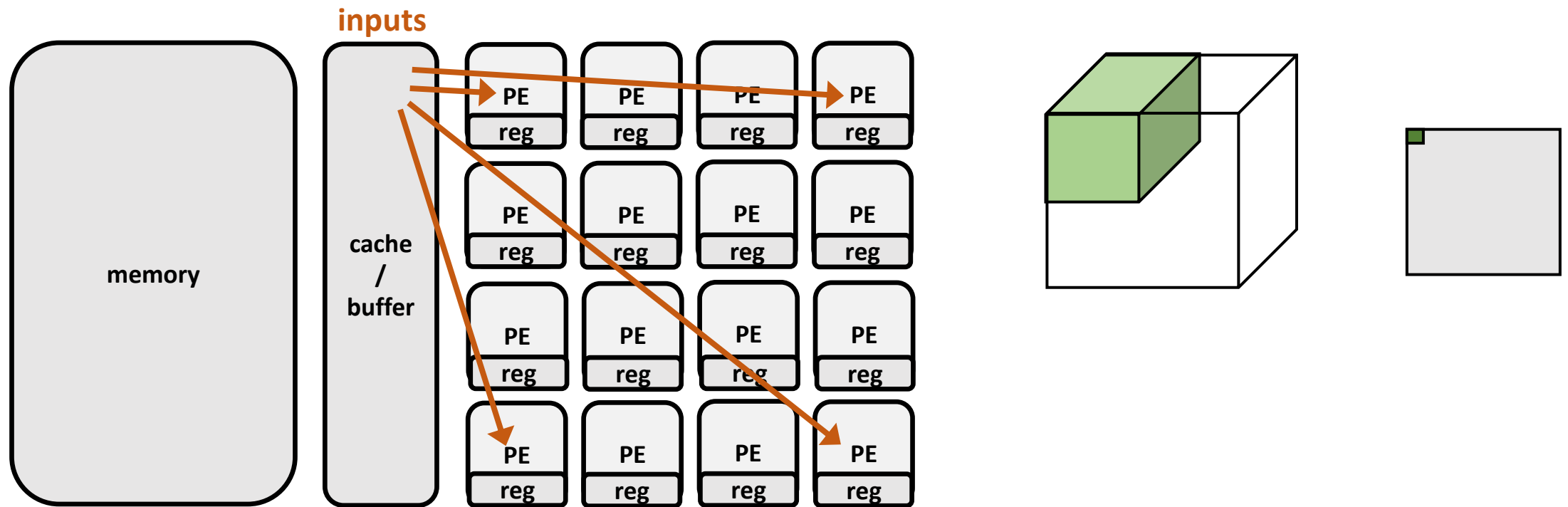
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

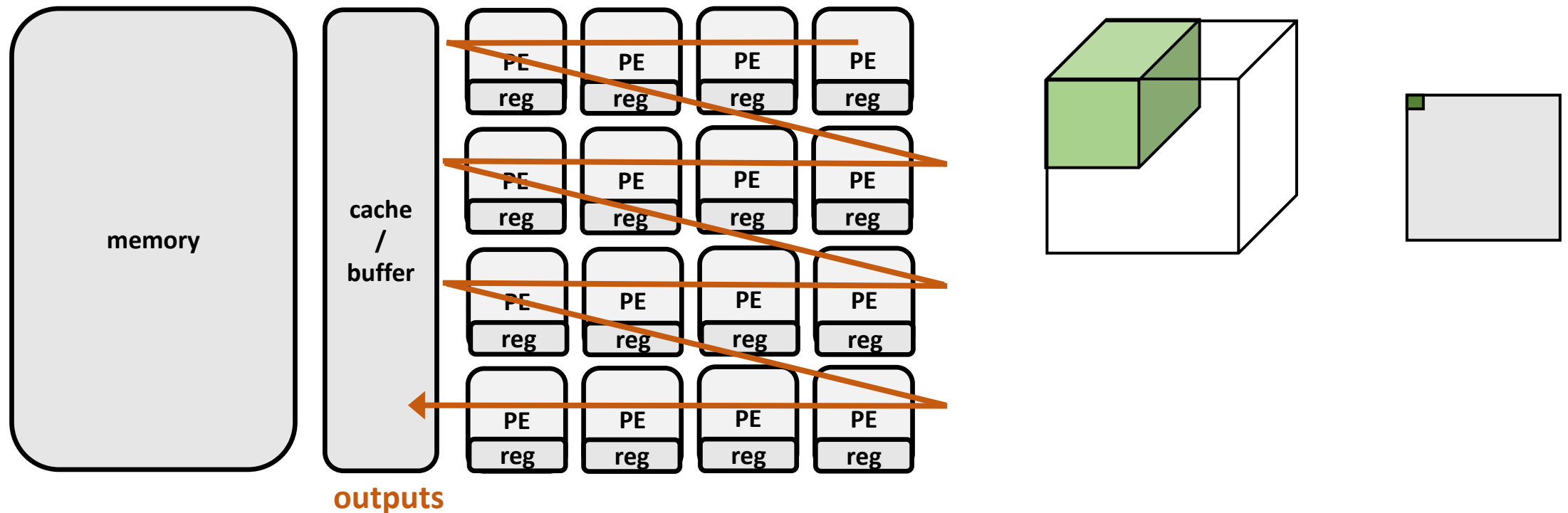
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

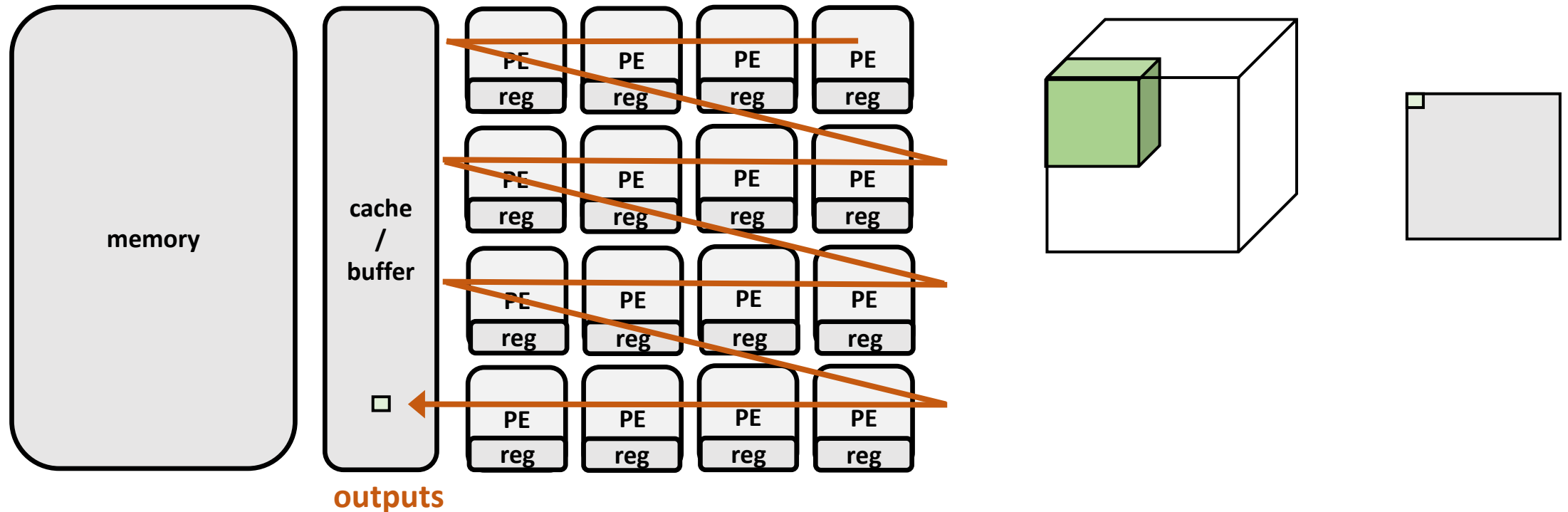
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

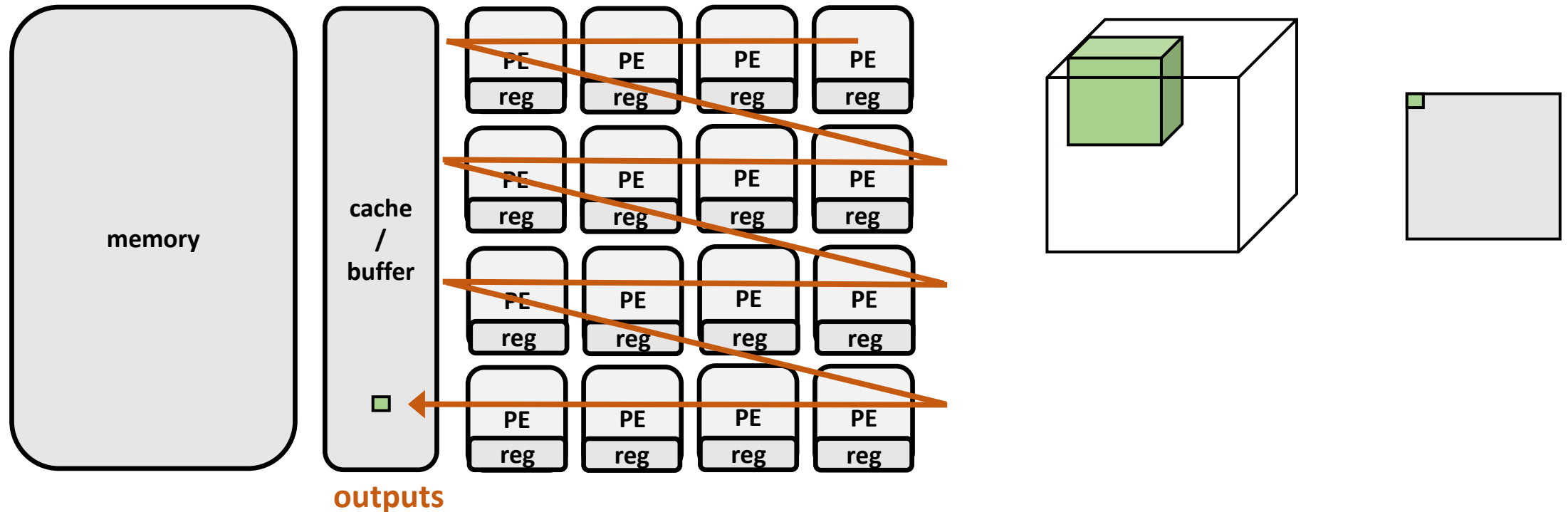
- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer

